

Control de Accionamientos y Automatización de Grúa Portacontenedores de Muelle tipo Pórtico

Proyecto Global Integrador

VERSIÓN FINAL

Aleo Rodrigo
Pucciarelli Nahuel

AUTÓMATAS Y CONTROL DISCRETO
Profesor Ing. Julián Gabriel

Universidad Nacional de Cuyo
Mendoza, Argentina

2026

Resumen

En este trabajo se realizará el desarrollo e implementación de un Sistema Híbrido de Control y Protección con Operación Semi-Automática coordinada para una Grúa Portacontenedores Portuaria de Muelle tipo Pórtico, para carga y descarga de contenedores en grandes barcos de transporte marítimo.

El sistema híbrido de control y protección incluirá 3 niveles jerárquicos:

- Nivel 2 - Control Regulatorio: estados continuos, muestreados en tiempo discretizado (Sampled CTDS).
- Nivel 1 - Control Supervisor: autómata, estados discretos activados por eventos (DEDS).
- Nivel 0 - Protección: en espera; actúa en situación de emergencia, lleva el sistema a estado seguro.

Índice

1. Modelado de la Planta	4
1.1. Subsistema del Carro (Trolley)	5
1.1.1. Dinámica del Carro	5
1.1.2. Acoplamiento Elástico del Cable	5
1.1.3. Dinámica del Accionamiento	6
1.1.4. Modelo de Espacio de Estados para Simulación	7
1.2. Subsistema de Izaje (Hoist)	8
1.2.1. Dinámica de la Carga Suspendida	8
1.2.2. Acoplamiento Elástico del Cable	10
1.2.3. Dinámica del Accionamiento	10
1.2.4. Modelo de Espacio de Estados para Simulación	11
1.3. Perfil 2D de Obstáculos	12
1.4. Instrumentación y Sensores de Posición	13
1.4.1. Sistema de Encoders	13
1.4.2. Fines de Carrera	14
1.5. Moduladores de Torque (Accionamientos)	14
1.5.1. Modulador del Carro	15
1.5.2. Modulador de Izaje	15
1.6. Modelo de la Planta en Simulink	16
2. Diseño del Control Regulatorio (Nivel 2)	16
2.1. Control de Izaje de la Carga	17
2.1.1. Modelo de Izaje para Control de Velocidad	17
2.1.2. Diseño del Lazo Regulador (Control PI de Velocidad)	17
2.1.3. Compensación de Gravedad (Feedforward)	18
2.1.4. Planificación Cinemática de Trayectorias	18
2.1.5. Saturación Dinámica por Límite de Potencia	18
2.2. Control de Traslación del Carro	18
2.2.1. Modelo del Carro para Control de Velocidad	19
2.2.2. Diseño del Lazo Regulador (Control PI de Velocidad)	19
2.2.3. Planificación Cinemática de Trayectorias	20
2.3. Diseño del Controlador de Balanceo (Anti-Sway)	20
2.3.1. Estructura Desacoplada	20
2.3.2. Deducción de la Ley de Control No Lineal	21
2.3.3. Sintonía por Linealización Local (Gain Scheduling)	21
2.4. Observadores de Estado	22
2.4.1. Cálculo de Ganancias	23
2.5. Modelo del Controlador Regulatorio en Simulink	24
3. Diseño del Control Supervisor (Nivel 1)	25
3.1. Autómata Principal de Modos de Operación	25
3.2. Gestión de Twistlocks	26
3.3. Suavizador de Consignas de Velocidad	27
3.4. Análisis Espacial, Perfil de Obstáculos y Actualización de Cruceta	27
3.5. Supervisión de Comunicación (Watchdog Heartbeat)	28
3.6. Límites Suaves	28
3.7. Modelo del Control Supervisor en Simulink	29
3.8. Control del Joystick y Botones Físicos	30
4. Diseño del Control de Seguridad (Nivel 0)	30

4.1. Autómata de Estados de Seguridad	30
4.2. Monitor de Comunicaciones	31
4.3. Modelo del Control de Seguridad en Simulink	32
5. Comunicación y Animación con Unity	33
6. CODESYS y ST	36
6.1. Control Regulatorio (Nivel 2) en CODESYS	37
6.2. Control Supervisor (Nivel 1) en CODESYS	38
6.3. Control de Seguridad (Nivel 0) en CODESYS	41
6.4. Comunicación OPC Simulink/CODESYS	42
6.5. Comunicación UDP Simulink/CODESYS	45
7. Generación Automática de Código ST	48
8. Conclusiones	51
9. Repositorio y Video Demostrativo	51

1. Modelado de la Planta

Este proyecto considera el control de accionamientos eléctricos y operación semi-automática coordinada de los dos movimientos principales continuos y simultáneos de la grúa en un plano vertical (2D): traslación horizontal del carro (Trolley) sobre las vigas principal y móvil (paralelo al eje x); e Izaje/descenso, o traslación vertical, de la carga (Hoist) bajo la posición del carro (paralelo al eje y).

Ambos movimientos son impulsados mediante cables de acero (wireropes) desde mecanismos rotativos con tambores, cajas de engranajes, frenos mecánicos y motores eléctricos, ubicados en una Casa de Máquinas (Machinery House) fija, con sus correspondientes accionamientos electrónicos de velocidad variable; dichos movimientos tienen *restricciones de operación*: posiciones límite de recorrido, velocidades máximas y aceleraciones máximas. Están *acoplados entre sí por la carga* (headblock/spreader con o sin container), que se balancea suspendida del carro que la traslada mediante los cables de izaje.

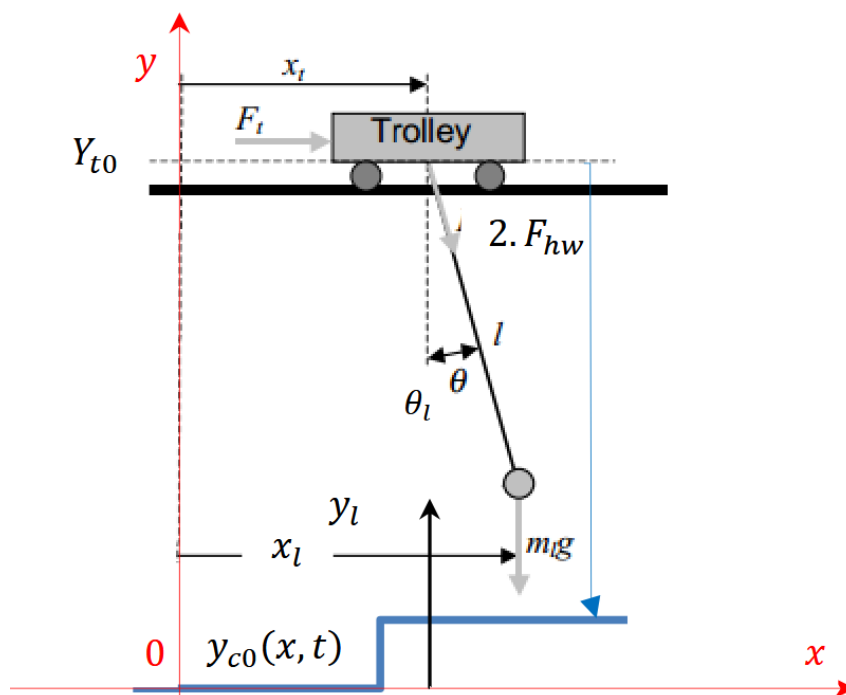


Figura 1: Diagrama básico del modelo.

Hipótesis simplificativas

Para el modelado dinámico de los movimientos de la grúa, se consideran las siguientes hipótesis simplificativas:

- **Estructura:** Se asume totalmente rígida; se desprecian las deformaciones y vibraciones de las vigas y el pórtico.
- **Mecanismo de carro:** Se modela como una transmisión rígida accionada por un cable equivalente siempre en tensión, considerando su elasticidad longitudinal y amortiguamiento.
- **Sistema de izaje:** Se simplifica a un sistema equivalente (un solo motor/tambor). El cable se considera de masa despreciable y comportamiento elástico solo a tracción (no soporta compresión).
- **Carga:** Se modela como una masa puntual concentrada en la base del spreader, con 2

grados de libertad en el plano vertical (2D).

- *Efectos externos*: Solo se consideran la gravedad y las fuerzas de contacto vertical (modelo elástico lineal). Se desprecia la resistencia aerodinámica frontal al viento de la carga y del carro en movimiento.

1.1. Subsistema del Carro (Trolley)

El sistema de traslación del carro se modela como un sistema electromecánico de dos masas (inercia rotacional del accionamiento y masa traslacional del carro) acopladas mediante un elemento flexible (cable de acero). A continuación, se presentan las ecuaciones fundamentales y su desarrollo para la simulación.

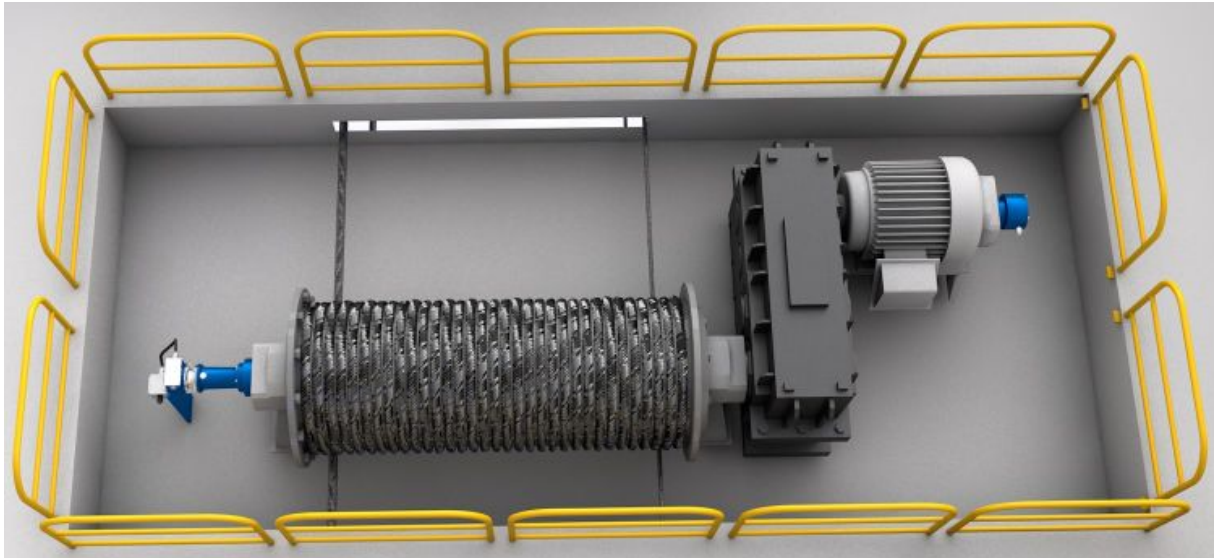


Figura 2: Diagrama del subsistema del carro (tambor-motor) (Trolley).

1.1.1. Dinámica del Carro

Aplicando la segunda ley de Newton al movimiento horizontal del carro, se tiene:

$$M_t \frac{dv_t(t)}{dt} = F_{tw}(t) - b_t v_t(t) + F_{coupling}(t) \quad (1)$$

Donde:

- M_t : Masa del carro.
- $F_{tw}(t)$: Fuerza de tracción ejercida por el cable.
- b_t : Coeficiente de fricción viscosa del carro sobre los rieles.
- $F_{coupling}(t) = 2F_{hw} \sin \theta_l$: Fuerza horizontal perturbadora debida al balanceo de la carga.

1.1.2. Acoplamiento Elástico del Cable

El cable transmite la fuerza entre el tambor y el carro. Se modela como un resorte con rigidez K_{tw} y amortiguación b_{tw} :

$$F_{tw}(t) = K_{tw} (x_{td}(t) - x_t(t)) + b_{tw} (v_{td}(t) - v_t(t)) \quad (2)$$

Donde x_{td} y v_{td} son la posición y velocidad lineal equivalente del cable en la superficie del tambor.

1.1.3. Dinámica del Accionamiento

- **Dinámica del Tambor (Eje Lento):** La dinámica rotacional del eje de salida (tambor) está dada por:

$$J_{td} \frac{d\omega_{td}(t)}{dt} = T_{td}(t) - b_{td}\omega_{td}(t) - T_{tdl}(t) \quad (3)$$

Donde $T_{tdl}(t) = F_{tw}(t) r_{td}$ es el torque de carga generado por la tensión del cable.

- **Dinámica del Motor (Eje Rápido):** La dinámica rotacional del eje de entrada (motor) está dada por:

$$J_{tm+tb} \frac{d\omega_{tm}(t)}{dt} = T_{tm}(t) + T_{tb}(t, BRK_t) - b_{tm}\omega_{tm}(t) - T_{tml}(t) \quad (4)$$

Donde $T_{tml}(t)$ es el torque resistente que la caja reductora demanda al motor.

Modelo del Freno Mecánico de Operación

El sistema dispone de un freno mecánico en el eje rápido del motor, configurado como Normal Cerrado (NC) por seguridad. El torque de frenado T_{tb} depende del estado lógico de la señal de control BRK_t (0: Desenergizado/Cerrado, 1: Energizado/Abierto).

Para evitar discontinuidades numéricas abruptas durante la simulación (chattering) al momento de la detención, se modela la fricción del freno cerrado utilizando una función tangente hiperbólica escalada, tal como sugiere la especificación técnica:

$$T_{tb}(t, BRK_t) = \begin{cases} 0 & \text{si } BRK_t = 1 \text{ (Abierto/Energizado)} \\ -T_{tb_Max} \tanh(b_{tb}\omega_{tm}(t)) & \text{si } BRK_t = 0 \text{ (Cerrado/Frenado)} \end{cases} \quad (5)$$

Donde T_{tb_Max} es el torque máximo de fricción estática del freno y b_{tb} es un coeficiente de pendiente elevada que aproxima el comportamiento de la fricción viscosa.

Modelo Equivalente Referido al Motor

Para la simulación y el diseño de control, es necesario unificar las ecuaciones (3) y (4) en una sola ecuación diferencial referida al eje del motor.

La caja reductora vincula el eje rápido y el lento mediante la relación de transmisión i_t :

$$\text{Velocidades: } \omega_{td}(t) = \frac{\omega_{tm}(t)}{i_t} \implies \frac{d\omega_{td}(t)}{dt} = \frac{1}{i_t} \frac{d\omega_{tm}(t)}{dt} \quad (6)$$

$$\text{Torques: } T_{td}(t) = T_{tml}(t) i_t \quad (7)$$

Sustituyendo las relaciones (6) y (7) en la ecuación del eje lento (3):

$$J_{td} \left(\frac{1}{i_t} \frac{d\omega_{tm}(t)}{dt} \right) = (T_{tml}(t) i_t) - b_{td} \left(\frac{\omega_{tm}(t)}{i_t} \right) - T_{tdl}(t)$$

Para despejar el torque de carga en el eje motor (T_{tml}), dividimos toda la ecuación por i_t :

$$\frac{J_{td}}{i_t^2} \frac{d\omega_{tm}(t)}{dt} = T_{tml}(t) - \frac{b_{td}}{i_t^2} \omega_{tm}(t) - \frac{T_{tdl}(t)}{i_t} \quad (8)$$

Despejando T_{tml} y sustituyéndolo en la ecuación del eje rápido (4), obtenemos la ecuación unificada:

$$\left(J_{tm+tb} + \frac{J_{td}}{i_t^2} \right) \frac{d\omega_{tm}(t)}{dt} = T_{tm}(t) + T_{tb}(t, \text{BRK}_t) - \left(b_{tm} + \frac{b_{td}}{i_t^2} \right) \omega_{tm}(t) - \frac{T_{td}(t)}{i_t} \quad (9)$$

Se definen los parámetros equivalentes totales:

$$J_{eq,t} = J_{tm+tb} + \frac{J_{td}}{i_t^2} \quad ; \quad b_{eq,t} = b_{tm} + \frac{b_{td}}{i_t^2} \quad (10)$$

1.1.4. Modelo de Espacio de Estados para Simulación

Finalmente, las ecuaciones diferenciales se ordenan despejando las derivadas de mayor orden para su implementación mediante bloques en Simulink.

- **Accionamiento del carro:**

$$\frac{d\omega_{tm}(t)}{dt} = \frac{1}{J_{eq,t}} \left[T_{tm}(t) + T_{tb}(t, \text{BRK}_t) - b_{eq,t} \omega_{tm}(t) - \frac{r_{td}}{i_t} F_{tw}(t) \right] \quad (11)$$

- **Dinámica del carro:**

$$\frac{dv_t(t)}{dt} = \frac{1}{M_t} [F_{tw}(t) - b_t v_t(t) + F_{coupling}(t)] \quad (12)$$

- **Posición del carro:**

$$\frac{dx_t(t)}{dt} = v_t(t) \quad (13)$$

- **Fuerza del cable del carro:**

$$F_{tw}(t) = K_{tw}(x_{td}(t) - x_t(t)) + b_{tw}(v_{td}(t) - v_t(t)) \quad (14)$$

Variables auxiliares:

- Velocidad lineal del tambor: $v_{td}(t) = \frac{r_{td}}{i_t} \omega_{tm}(t)$
- Posición lineal del tambor: $x_{td}(t) = \int v_{td}(t) dt$

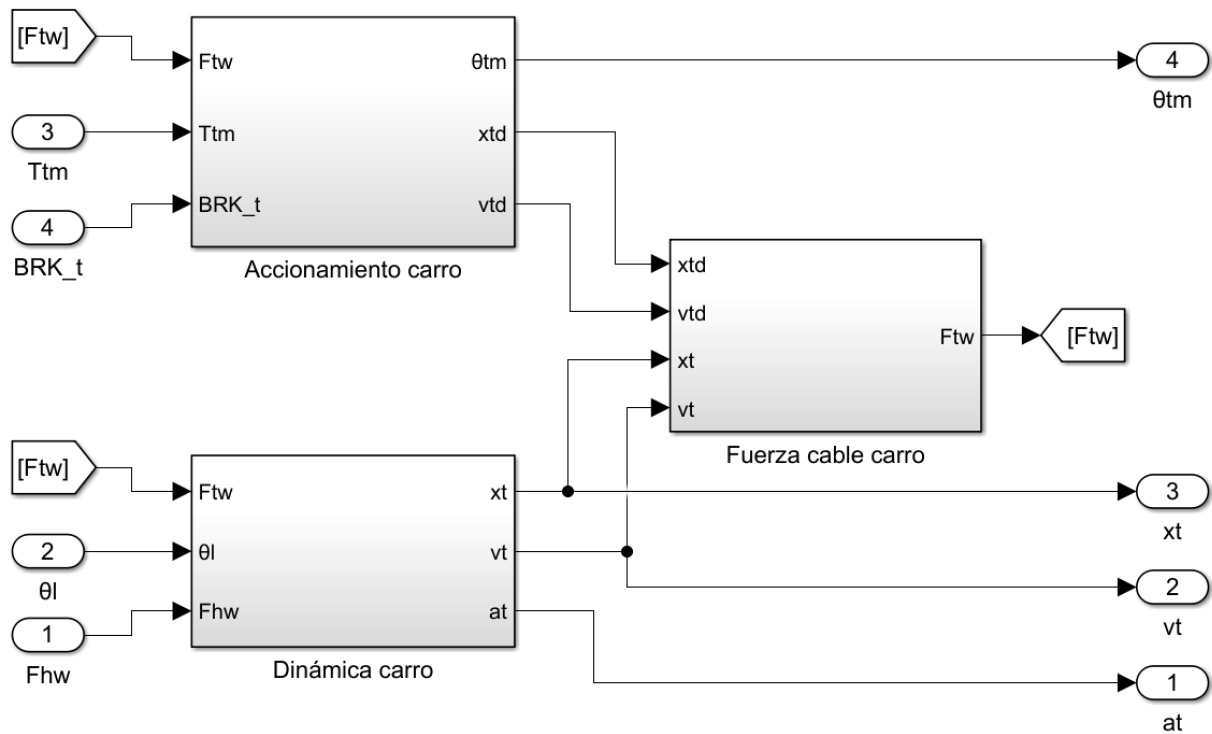


Figura 3: Modelo del carro en Simulink (Trolley).

1.2. Subsistema de Izaje (Hoist)

El sistema de izaje controla la elevación de la carga mediante un mecanismo de tambor accionado por motor eléctrico, con un sistema de cableado (reeving) que introduce una relación de transmisión mecánica adicional.

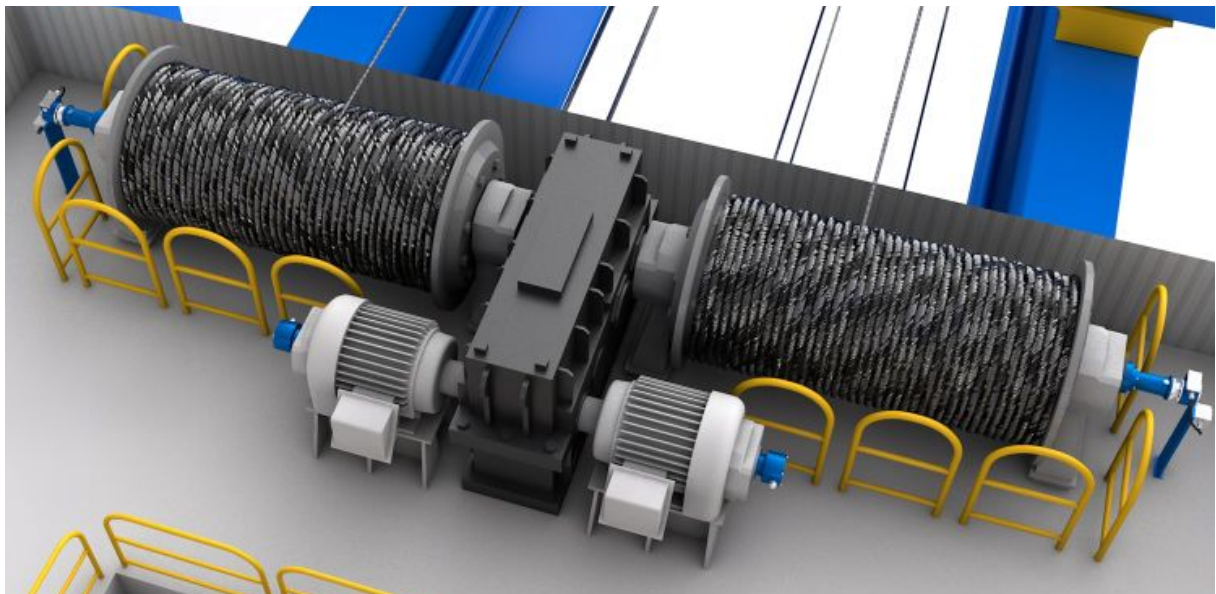


Figura 4: Diagrama del subsistema de izaje (tambores-motores) (Hoist).

1.2.1. Dinámica de la Carga Suspendeda

La carga se modela como una masa puntual concentrada en el centro de gravedad del conjunto Spreader/Contenedor, con 2 grados de libertad: traslación horizontal x_l y vertical y_l . Su dinámica

es híbrida, ya que sus parámetros (masa y geometría) cambian discretamente según el estado de los twistlocks y su interacción con el entorno cambia según si está suspendida o apoyada.

Definición de Masa y Geometría

Los parámetros inerciales y geométricos dependen del estado discreto de los cerrojos (twistlocks), controlado por la señal lógica *TLK*. Según la especificación:

- **Masa Total (m_l):**

$$m_l(\textit{TLK}) = \begin{cases} M_s & \text{si } \textit{TLK} = \text{Off (solo spreader)} \\ M_s + M_{cX} & \text{si } \textit{TLK} = \text{On (spreader + contenedor)} \end{cases} \quad (15)$$

- **Altura de la Carga (h_c):** Define la distancia desde el punto de suspensión hasta la base inferior (necesaria para calcular colisiones/apoyo).

$$h_c(\textit{TLK}) = \begin{cases} 0 & \text{si } \textit{TLK} = \text{Off} \\ H_c & \text{si } \textit{TLK} = \text{On} \end{cases} \quad (16)$$

Ecuaciones de Movimiento

Aplicando la segunda ley de Newton en el plano vertical, la aceleración de la carga resulta de la suma vectorial de: la tensión del cable, la gravedad y las fuerzas de reacción del entorno.

- **Dinámica Horizontal (Ecuación de Balanceo):** La componente horizontal de la tensión del cable actúa como fuerza restauradora hacia la posición del carro.

$$m_l(\textit{TLK}) \frac{d v_{lx}(t)}{dt} = -2F_{hw}(t) \sin \theta_l(t) + F_{cx}(t) \quad (17)$$

Nota: El signo negativo indica que si la carga está a la derecha ($\theta > 0$), la fuerza tira hacia la izquierda.

- **Dinámica Vertical (Ecuación de Izaje):**

$$m_l(\textit{TLK}) \frac{d v_{ly}(t)}{dt} = 2F_{hw}(t) \cos \theta_l(t) - m_l(\textit{TLK}) g + F_{cy}(t) \quad (18)$$

Interacción con el Entorno (Modelo de Contacto)

La carga interactúa con un perfil de obstáculos $y_{c0}(x, t)$ (suelo, barco, otros contenedores). Se define la penetración vertical δ_y como:

$$\delta_y(t) = y_{c0}(x_l, t) - (y_l(t) - h_c(\textit{TLK}))$$

Las fuerzas de reacción actúan solo cuando hay contacto ($\delta_y \geq 0$):

- **Fuerza Vertical de Soporte (F_{cy}):** Modelo elástico-amortiguado (resorte K_{cy} y amortiguador b_{cy}) que resiste la compresión.

$$F_{cy}(t) = \begin{cases} K_{cy} \delta_y(t) - b_{cy} v_{ly}(t) & \text{si } \delta_y(t) \geq 0 \text{ (Apoyado)} \\ 0 & \text{si } \delta_y(t) < 0 \text{ (Suspendido)} \end{cases} \quad (19)$$

- **Fuerza Horizontal de Fricción (F_{cx}):** Resistencia al arrastre horizontal, existente solo si hay contacto vertical.

$$F_{cx}(t) = \begin{cases} -b_{cx} v_{lx}(t) & \text{si } \delta_y(t) \geq 0 \\ 0 & \text{si } \delta_y(t) < 0 \end{cases} \quad (20)$$

1.2.2. Acoplamiento Elástico del Cable

El cable de izaje se modela como un resorte-amortiguador. A diferencia del carro, su longitud desenrollada $l_h(t)$ cambia drásticamente, lo que afecta su rigidez y amortiguamiento.

Tanto la rigidez unitaria k_{hwu} como el amortiguamiento unitario b_{hwu} escalan con la longitud total del cable bajo tensión ($2l_h(t) + L_{h0}$):

$$K_{hw}(l_h) = \frac{k_{hwu}}{2l_h(t) + L_{h0}} \quad ; \quad b_{hw}(l_h) = b_{hwu} (2l_h(t) + L_{h0}) \quad (21)$$

La fuerza de tensión F_{hw} se calcula solo si el cable está tenso ($l \geq l_h$):

$$F_{hw}(t) = \begin{cases} K_{hw}(l_h) \Delta L + b_{hw}(l_h) \Delta V & \text{si } l(t) \geq l_h(t) \text{ (Tenso)} \\ 0 & \text{si } l(t) < l_h(t) \text{ (Flojo/Slack)} \end{cases} \quad (22)$$

Donde la elongación es $\Delta L = 2(l(t) - l_h(t))$ debido al reeving.

1.2.3. Dinámica del Accionamiento

El sistema posee dos frenos y una transmisión.

- **Eje Lento (Tambor):** Posee el Freno de Emergencia (T_{hEb}).

$$J_{hd+hEb} \frac{d\omega_{hd}(t)}{dt} = T_{hd}(t) + T_{hEb}(\text{BRK}_{hE}) - b_{hd} \omega_{hd}(t) - T_{hdl}(t) \quad (23)$$

- **Eje Rápido (Motor):** Posee el Freno de Operación (T_{hb}).

$$J_{hm+hB} \frac{d\omega_{hm}(t)}{dt} = T_{hm}(t) + T_{hb}(\text{BRK}_h) - b_{hm} \omega_{hm}(t) - T_{hml}(t) \quad (24)$$

Relación cinemática del Reeving (2:1) y Tambor:

$$v_h(t) = \frac{r_{hd} \omega_{hd}(t)}{2} = -\frac{dl_h(t)}{dt} \quad (25)$$

El factor 2 divide la velocidad tangencial del tambor porque es un aparejo móvil.

Modelo de los Frenos Mecánicos de Operación y Emergencia

A diferencia del carro, el sistema de izaje cuenta con dos sistemas de frenado independientes para garantizar la seguridad de la carga suspendida. Ambos operan bajo lógica "Normal Cerrado" (NC).

- **Freno de Operación (Eje Rápido):** Ubicado en el eje del motor, actúa durante las paradas normales y el mantenimiento de posición (parking).

$$T_{hb}(t, \text{BRK}_h) = \begin{cases} 0 & \text{si } \text{BRK}_h = 1 \text{ (Energizado/Abierto)} \\ -T_{hb_Max} \tanh(b_{hb} \omega_{hm}(t)) & \text{si } \text{BRK}_h = 0 \text{ (Desenergizado/Cerrado)} \end{cases} \quad (26)$$

- **Freno de Emergencia (Eje Lento):** Ubicado directamente sobre el tambor, actúa solo ante fallas críticas (Nivel 0 de Protección) o parada de emergencia.

$$T_{hEb}(t, \text{BRK}_{hE}) = \begin{cases} 0 & \text{si } \text{BRK}_{hE} = 1 \text{ (Energizado/Abierto)} \\ -T_{hEb_Max} \tanh(b_{hEb} \omega_{hd}(t)) & \text{si } \text{BRK}_{hE} = 0 \text{ (Desenergizado/Cerrado)} \end{cases} \quad (27)$$

Modelo Equivalente Referido al Motor

Dado que el Freno de Emergencia está en el eje lento, su torque también se divide por la relación de transmisión i_h al reflejarlo.

$$J_{eq,h} = J_{hm+hb} + \frac{J_{hd+hEb}}{i_h^2} \quad ; \quad b_{eq,h} = b_{hm} + \frac{b_{hd}}{i_h^2} \quad ; \quad T_{hEb,ref} = \frac{T_{hEb}}{i_h} \quad (28)$$

1.2.4. Modelo de Espacio de Estados para Simulación

Finalmente, las ecuaciones diferenciales se ordenan despejando las derivadas de mayor orden para su implementación mediante bloques en Simulink.

■ Accionamiento del izaje:

$$\frac{d\omega_{hm}(t)}{dt} = \frac{1}{J_{eq,h}} \left[T_{hm}(t) + T_{hb}(t, \text{BRK}_h) + \frac{T_{hEb}(t, \text{BRK}_{hE})}{i_h} - b_{eq,h} \omega_{hm}(t) - \frac{r_{hd}}{i_h} F_{hw}(t) \right] \quad (29)$$

donde $T_{hdl} = F_{hw} r_{hd}$ es el torque ejercido por el cable sobre el tambor.

■ Dinámica de la carga:

$$\frac{dv_{ly}(t)}{dt} = \frac{1}{m_l(\text{TLK})} [2 F_{hw}(t) \cos \theta_l - m_l(\text{TLK}) g + F_{cy}(t)] \quad (30)$$

$$\frac{dv_{lx}(t)}{dt} = \frac{1}{m_l(\text{TLK})} [-2 F_{hw}(t) \sin \theta_l(t) + F_{cx}(t)] \quad (31)$$

■ Posición de la carga:

$$\frac{dy_l(t)}{dt} = v_{ly}(t) \quad (32)$$

■ Fuerza del cable de izaje:

$$F_{hw}(t) = \begin{cases} 2K_{hw}(l_h) (l(t) - l_h(t)) + 2b_{hw}(l_h) \left(\frac{dl(t)}{dt} - \frac{dl_h(t)}{dt} \right), & \text{si } l \geq l_h \text{ (Tenso)} \\ 0, & \text{si } l < l_h \text{ (Flojo)} \end{cases} \quad (33)$$

Nota: Se utiliza $\frac{dl}{dt}$ en lugar de v_{ly} para considerar el estiramiento inducido por el balanceo ($\theta_l \neq 0$).

■ Longitud de cable desplegado:

$$\frac{dl_h(t)}{dt} = - \frac{r_{hd}}{2i_h} \omega_{hm}(t) \quad (34)$$

El signo negativo indica que una velocidad positiva del motor ($\omega_{hm} > 0$) enrolla el cable y reduce la longitud l_h .

Variables auxiliares:

- Rigidez variable: $K_{hw}(l_h) = \frac{k_{hwu}}{2l_h(t) + L_{h0}}$
- Amortiguamiento variable: $b_{hw}(l_h) = b_{hwu} (2l_h(t) + L_{h0})$
- Longitud geométrica actual (Pitágoras): $l(t) = \sqrt{(x_l(t) - x_t(t))^2 + (Y_{t0} - y_l(t))^2}$ (Y_{t0} es la altura fija de poleas de izaje en el carro)

- Velocidad de cambio de longitud (para el amortiguamiento):

$$\frac{dl(t)}{dt} = \frac{(x_l(t) - x_t(t))(v_{lx}(t) - v_t(t)) - (Y_{t0} - y_l(t))v_{ly}(t)}{l(t)}$$

- Cálculo del ángulo de balanceo:

$$\theta_l = \text{atan2} \left(\frac{x_l(t) - x_t(t)}{Y_{t0} - y_l(t)} \right)$$

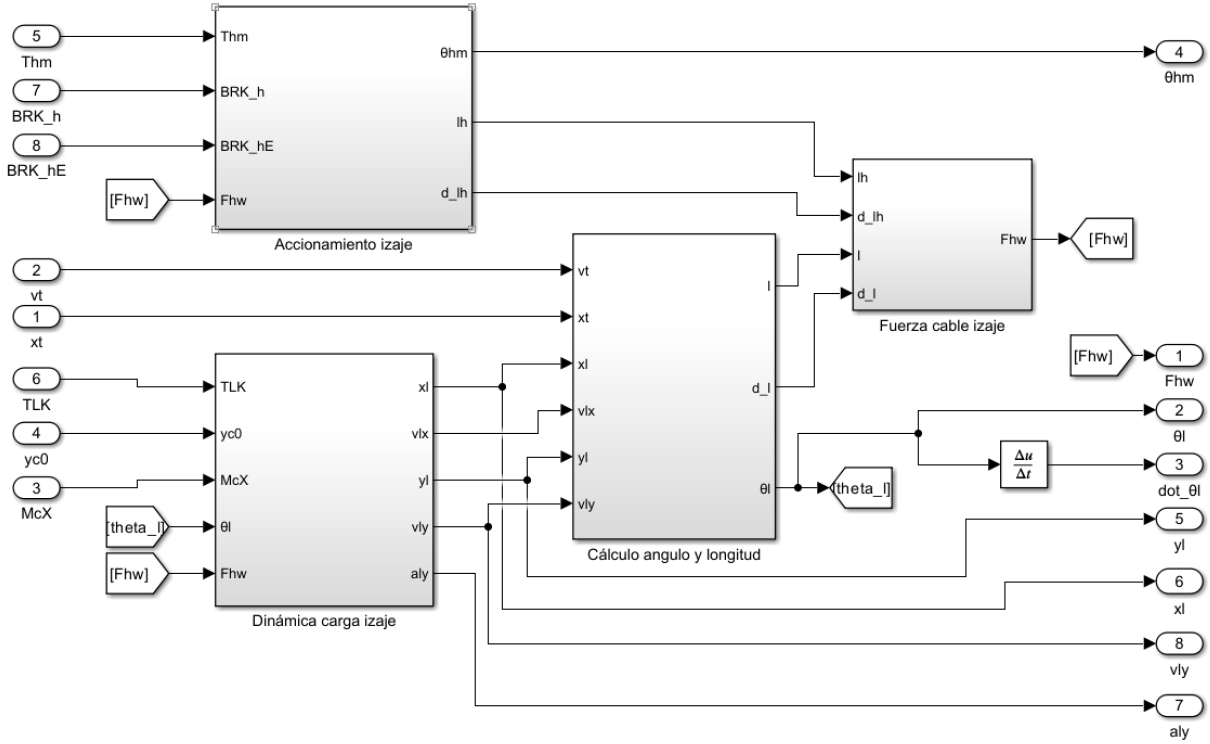


Figura 5: Modelo del izaje en Simulink (Hoist).

1.3. Perfil 2D de Obstáculos

El carro cuenta con un sistema de sensado LIDAR (escaner láser infrarojo 1D) para relevar el perfil topográfico de obstáculos y_{c0} en el área de operación. Este perfil define la superficie efectiva con la que puede interactuar la carga suspendida (spreader + contenedor). El sensor está colocado en el carro, 3m hacia adelante del spreader, y mide la distancia vertical instantánea desde el carro en movimiento hasta la pila de obstáculos.

Se puede discretizar la posición del carro (y por lo tanto del spreader) en el rango $x_t \in [x_{tmin}, x_{tmax}]$ utilizando el ancho del contenedor W_c como paso entre centros. Luego, la cantidad de posiciones discretas a lo largo de x_t :

$$N_{x_t} = \text{floor} \left(\frac{x_{tmax} - x_{tmin}}{W_c} \right) + 1 \quad (35)$$

Según la especificación se tiene que $x_{tmin} = -30$ m (sobre el muelle) y $x_{tmax} = 50$ m (sobre el barco), y que $W_c = 2,44$ m (ancho de container estándar ISO), se obtiene que $N_{x_t} = 33$. Los posibles índices serán entonces:

$$i = 1, 2, 3, \dots, 33 \quad (36)$$

Luego, la última posición discreta corresponde a:

$$x_{33} = -30 \text{ m} + (33 - 1) \cdot 2,44 \text{ m} = 48,08 \text{ m} \quad (37)$$

Por lo que se verifica que el rango de discretización cubre todo el área de operación del carro sin exceder el límite de $x_{t_{max}}$.

Para simular el sensado del terreno, se crea un vector h_{vec} de tamaño N_{x_t} que es rellenado con valores de altura representativos del perfil de relieve del terreno real. De esa manera, a medida que el carro se desplaza, las mediciones tomadas por el sensor LIDAR se representan mediante la extracción de los valores de h_{vec} y su guardado en un vector y_{c0_vec} también de tamaño N_{x_t} , e inicialmente nulo. Para saber si una posición ya ha sido escaneada, en memoria se almacena además un vector $is_scanned_vec$ también de tamaño N_{x_t} con valores booleanos que indican si la posición correspondiente ya ha sido sensada o no. De esta manera, el sistema de control luego puede tomar diferentes decisiones según si el perfil de obstáculos por donde debe pasar el carro ya ha sido escaneado o no.

Se puede calcular el índice i correspondiente a la posición actual del sensor (teniendo en cuenta el offset respecto al carro) con:

$$i = \text{floor} \left(\frac{(x_t(t) + 3 \text{ m}) - x_{t_{min}}}{W_c} \right) \quad (38)$$

El vector y_{c0_vec} se actualiza dinámicamente durante la operación de la grúa, permitiendo que el sistema de control y protección utilice el perfil de obstáculos más reciente para planificar movimientos seguros y evitar colisiones.

1.4. Instrumentación y Sensores de Posición

Para la retroalimentación del sistema de control y la protección automática, se modelan sensores de posición continua (encoders) y discretos (fines de carrera).

1.4.1. Sistema de Encoders

Conforme a la especificación, los sensores de posición son encoders absolutos o incrementales montados solidarios al eje de cada motor. El sistema de control no recibe los ángulos directamente, sino que procesa estas señales para obtener las posiciones lineales equivalentes.

1. **Encoder del Carro:** El sensor mide el ángulo acumulado del motor de carro, $\theta_{tm}(t)$. La señal de posición lineal se obtiene aplicando la relación de transmisión i_t y el radio del tambor r_{td} :

$$x_{enc}(t) = \theta_{tm}(t) \frac{r_{td}}{i_t} \quad (39)$$

2. **Encoder de Izaje:** El sensor mide el ángulo acumulado del motor de izaje, $\theta_{hm}(t)$. Se asume que el sistema se inicializa (homing) tal que $\theta_{hm} = 0$ corresponde al nivel del muelle ($y = 0$). Dado que un giro positivo del motor reduce la longitud del cable (l_h) y por ende aumenta la altura de la carga (y), la señal de retroalimentación $y_{enc}(t)$ se obtiene mediante:

$$y_{enc}(t) = \theta_{hm}(t) \left(\frac{r_{hd}}{2 i_h} \right) \quad (40)$$

Esta señal representa la altura operativa de la carga.

1.4.2. Fines de Carrera

La lógica de detección de límites se modela según la ubicación física descrita en las especificaciones, diferenciando entre sensores externos (viga) y sensores internos (tambor).

1. **Fines de Carrera de Carro (Fijos en Viga):** Al estar montados físicamente sobre la estructura de la grúa, estos sensores detectan la posición real de la masa del carro $x_t(t)$, independientemente de la elongación del cable o la posición del motor.

- **Entrada:** Posición real del carro x_t .
- **Lógica de activación:**

$$FC_{t,derecha} = \begin{cases} 0 & \text{si } x_t(t) < x_{t,max} \\ 1 & \text{si } x_t(t) \geq x_{t,max} \quad (\text{Operativo}) \\ 1 & \text{si } x_t(t) \geq x_{t,max} + 0,5m \quad (\text{Emergencia}) \end{cases} \quad (41)$$

(Análogamente para el retroceso (izquierda) hacia $x_{t,min}$).

2. **Fines de Carrera de Izaje (Rotativos en Tambor):** Al estar acoplados mecánicamente al eje del tambor, estos sensores detectan la cantidad de cable desenrollado. No miden la posición real de la carga (que varía con el balanceo).

- **Entrada:** Altura calculada por encoder $y_{enc}(t)$.
- **Lógica de activación:**

$$FC_{h,superior} = \begin{cases} 0 & \text{si } y_{enc}(t) < y_{h,max} \quad (\text{Operativo}) \\ 1 & \text{si } y_{enc}(t) \geq y_{h,max} \quad (\text{Operativo}) \\ 1 & \text{si } y_{enc}(t) \geq y_{h,max} + 0,5m \quad (\text{Emergencia}) \end{cases} \quad (42)$$

(Análogamente para el descenso hacia $y_{h,min}$).

1.5. Moduladores de Torque (Accionamientos)

En la simulación de la planta física, no es posible aplicar la señal de control de referencia directamente como fuerza mecánica instantánea. Los accionamientos reales (variadores de frecuencia y motores eléctricos) presentan limitaciones tanto en su capacidad de torque máximo como en su velocidad de respuesta dinámica. Para modelar este comportamiento, se introducen bloques moduladores de torque para cada eje.

Estos moduladores cumplen tres funciones principales:

1. **Lógica de frenos:** Anulan la entrega de torque del motor si los frenos mecánicos de retención se encuentran aplicados.
2. **Saturación:** Limitan la magnitud del comando de torque para proteger el motor y respetar sus curvas de operación nominales.
3. **Dinámica de primer orden:** Modelan el retardo electromagnético del motor mediante una constante de tiempo (τ), que representa la inercia eléctrica del sistema.

La dinámica general de los motores se representa mediante una ecuación diferencial de primer orden que vincula el torque limitado comandado (T_{Lim}^*) con el torque físico real entregado por el motor ($T(t)$):

$$\tau \cdot \frac{dT(t)}{dt} = T_{Lim}^*(t) - T(t)$$

Dado que la planta física es un sistema continuo, pero recibe señales de un controlador digital discreto, el valor comandado se mantiene constante mediante un retenedor de orden cero (ZOH) durante cada intervalo de muestreo.

1.5.1. Modulador del Carro

Para el sistema de traslación del carro, la saturación del motor posee un límite fijo (T_{tm_Max}). Previo a la saturación, la señal de referencia digital (T_{tm_ref}) se multiplica por el estado lógico del freno (BRK_t), obteniendo el comando efectivo T_{tm}^* . La ecuación de saturación por tramos de tiempo está dada por:

$$T_{tm_Lim}^*(k T_{s2} \leq t < (k + 1) T_{s2}) = \min(+T_{tm_Max}, \max(-T_{tm_Max}, T_{tm}^*(k T_{s2})))$$

T_{s2} representa el tiempo de muestreo del controlador regulatorio (Nivel 2) a desarrollar, y k es el índice del intervalo de muestreo actual.

1.5.2. Modulador de Izaje

El mecanismo de izaje presenta una complejidad mayor debido a la redundancia de seguridad (freno de operación BRK_h y freno de emergencia BRK_{hE}) y a la naturaleza de la curva de potencia del motor.

El comando de torque efectivo requiere que ambos frenos estén liberados simultáneamente:

$$T_{hm}^* = T_{hm_ref} BRK_h BRK_{hE}$$

A diferencia del carro, el límite de torque del motor de izaje no es constante. Para evitar exceder la potencia nominal del equipo a altas velocidades (zona de debilitamiento de campo o de potencia constante), el límite máximo permitido decae en función de la velocidad angular actual (ω_{hm}), comparada con su velocidad nominal (ω_{hm_rated}). La restricción dinámica se modela como:

$$T_{hm_Max_dyn}(t) = T_{hm_Max} \cdot \min\left(1, \left(\frac{\omega_{hm_rated}}{\omega_{hm}(t)}\right)^2\right)$$

Finalmente, la saturación aplicada al accionamiento del izaje en cada instante de muestreo discreto resulta en:

$$T_{hm_Lim}^*(k T_{s2} \leq t < (k + 1) T_{s2}) = \min(+T_{hm_Max_dyn}(t), \max(-T_{hm_Max_dyn}(t), T_{hm}^*(k T_{s2})))$$

1.6. Modelo de la Planta en Simulink

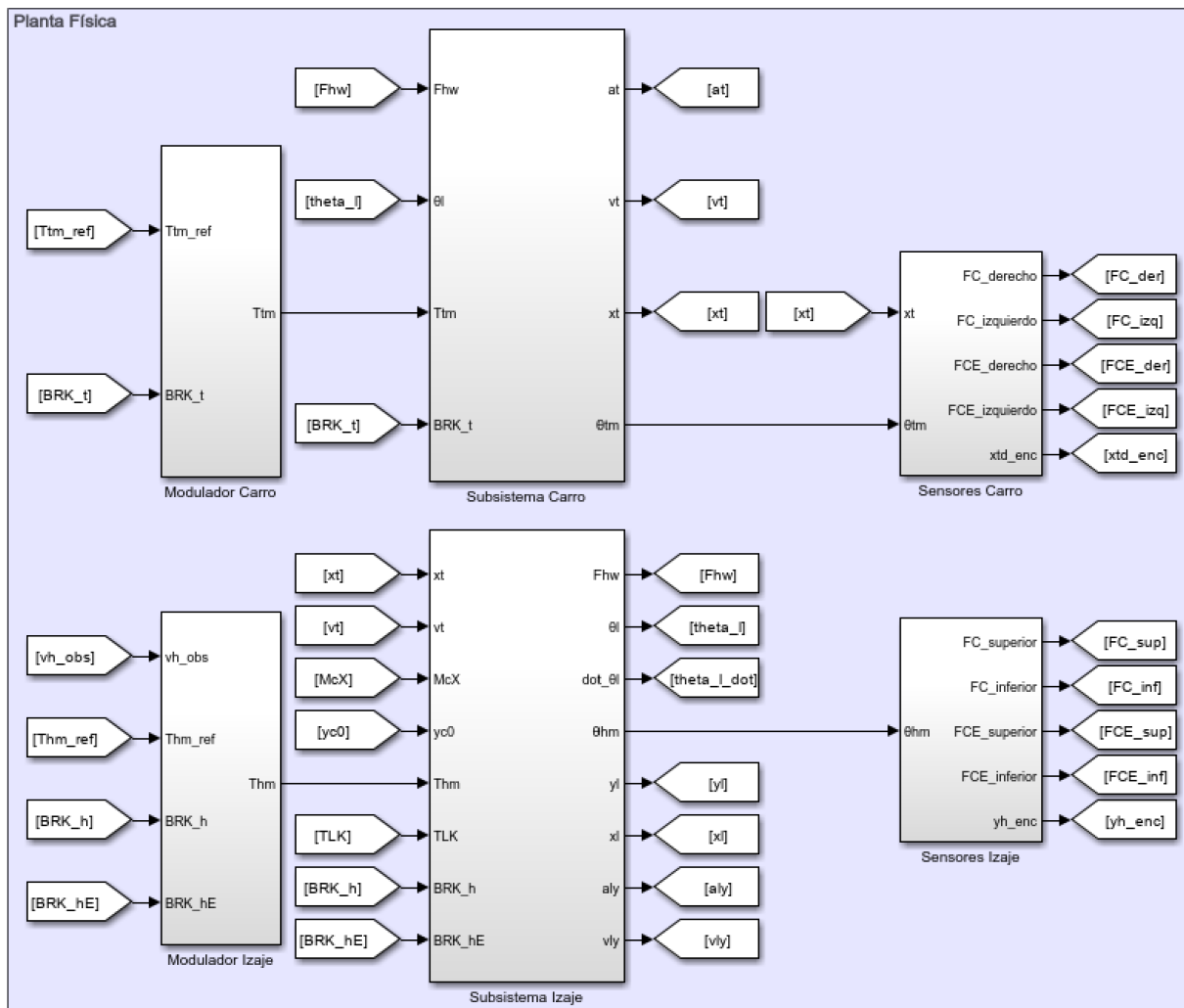


Figura 6: Modelo de la Planta en Simulink.

Integrando todos los elementos descritos, se construye un modelo completo de la planta física en Simulink. Este modelo incluye bloques para cada subsistema, los moduladores de torque y los sensores de posición y fines de carrera. La interconexión de estos bloques permite simular la dinámica acoplada del sistema y evaluar el desempeño del controlador regulatorio a desarrollar a continuación.

2. Diseño del Control Regulatorio (Nivel 2)

El Nivel 2 corresponde al control regulatorio de las variables continuas del sistema. Su objetivo es garantizar que los accionamientos sigan referencias de posición, velocidad y aceleración con precisión y suavidad, respetando restricciones físicas (límites de recorrido, saturaciones de torque, corrientes máximas) y atenuando perturbaciones como el balanceo de la carga o variaciones de masa. Este nivel opera en tiempo discretizado mediante muestreo (Sampled CTDS) y se implementará con lazos de realimentación en cascada (corriente/torque, velocidad y posición), además de compensaciones para mejorar la respuesta transitoria.

En este proyecto, el control regulatorio se encarga de los subsistemas de traslación del carro y de izaje, coordinando sus dinámicas para lograr movimientos simultáneos y seguros. Las señales

generadas en este nivel son consignas y comandos de accionamiento que luego son habilitados o bloqueados por el Control Supervisor (Nivel 1) y, ante eventos críticos, anulados por el Nivel de Protección (Nivel 0). De este modo, el Nivel 2 provee el desempeño continuo requerido, mientras los niveles superiores garantizan la lógica de operación y la seguridad global del sistema.

2.1. Control de Izaje de la Carga

Para el control de la carga en el izaje, se diseñó una arquitectura de Control Directo de Velocidad con Planificación Cinemática (Trayectoria $\rightarrow$ Velocidad $\rightarrow$ Torque). Esta estrategia evita los sobreesfuerzos mecánicos, disminuye el chattering y optimiza los tiempos de ciclo.

2.1.1. Modelo de Izaje para Control de Velocidad

Partiendo de la ecuación dinámica del izaje y considerando el sistema linealizado alrededor de un punto de operación, se asume que la dinámica del cable es rígida.

La ecuación de balance de torques en el eje del motor es:

$$J_{eq,h} \frac{d\omega_{hm}(t)}{dt} + b_{eq,h} \omega_{hm}(t) = T_{hm}(t) - T_{dist}(t) \quad (43)$$

Donde $T_{dist}(t)$ es el torque perturbador que encapsula todas las dinámicas no lineales y externas al accionamiento:

$$T_{dist}(t) = \underbrace{\frac{F_{hw}(t) r_{hd}}{i_h}}_{\text{Tensión de Cable}} + \underbrace{T_{hb}(t, BRK_h) + \frac{T_{hEb}(t, BRK_{hE})}{i_h}}_{\text{Fricción de Frenos}} \quad (44)$$

Aplicando la Transformada de Laplace, la función de transferencia de la planta es:

$$G_{vel}(s) = \frac{\Omega_{hm}(s)}{T_{hm}(s)} = \frac{1}{J_{eq,h} s + b_{eq,h}} \quad (45)$$

El polo mecánico del sistema es $P_{mec} = -b_{eq,h}/J_{eq,h}$. Dado que la fricción es muy baja en estos sistemas, el polo se encuentra muy cerca del origen ($P_{mec} \approx 0$), comportándose casi como un integrador puro.

2.1.2. Diseño del Lazo Regulador (Control PI de Velocidad)

Se utiliza un controlador Proporcional-Integral (PI) para asegurar error nulo de velocidad ante escalones de referencia y rechazo de perturbaciones de carga constantes.

$$C_{vel}(s) = K_p + \frac{K_i}{s} \quad (46)$$

La función de transferencia a lazo cerrado $T(s)$ resulta:

$$T(s) = \frac{K_p s + K_i}{J_{eq,h} s^2 + (b_{eq,h} + K_p) s + K_i} \quad (47)$$

Mediante el método de asignación de polos, se compara la ecuación característica con un polinomio de segundo orden deseado ($s^2 + 2\zeta\omega_n s + \omega_n^2$). Como $b_{eq,h}$ es muy pequeña en comparación con la inercia, las ecuaciones de diseño se simplifican:

$$K_p \approx 2\zeta\omega_n J_{eq,h} \quad ; \quad K_i = \omega_n^2 J_{eq,h} \quad (48)$$

Se selecciona $\omega_n \approx 8$ rad/s para asegurar rapidez sin excitar la resonancia del cable, y $\zeta \in [0,707, 1]$ para evitar sobrepasos. Esta sintonía garantiza la “rigidez” necesaria para sostener la carga.

El torque computado por el controlador PI en tiempo discreto es:

$$T_{PID}(k) = K_p e_{vel}(k) + K_i \sum_{j=1}^k \frac{T_s}{2} [e_{vel}(j) + e_{vel}(j-1)] \quad (49)$$

donde $e_{vel}(k) = \omega_{ref}(k) - \omega_{hm}(k)$.

2.1.3. Compensación de Gravedad (Feedforward)

Para mejorar el desempeño, se añade un término de pre-alimentación ($T_{g,ff}$) a la salida del PID. Esto permite que el término integral (K_i) solo corrija errores de modelo, delegando el esfuerzo de sostener el peso estático al compensador.

$$T_{g,ff} = \frac{m_l(\text{TLK}) g r_{hd}}{2 i_h} \quad (50)$$

La ley de control del lazo regulador es: $T_{ref}(k) = T_{PID}(k) + T_{g,ff}$.

2.1.4. Planificación Cinemática de Trayectorias

En lugar de un lazo de posición tradicional que persigue una coordenada espacial, la referencia de velocidad $\omega_{ref}(k)$ es dictada por un algoritmo supervisor de trayectorias en el control Supervisor. Este planificador evalúa el estado del entorno (obstáculos, alturas de crucero) y genera una velocidad requerida (v_{req}) basada en una ley PD hacia un objetivo dinámico $y_{target}(k)$:

$$v_{req}(k) = K_p (y_{target}(k) - y_{real}(k)) + K_d v_y(k) \quad (51)$$

El valor de y_{target} es gestionado por un autómata que garantiza la evasión de colisiones antes de habilitar el descenso.

2.1.5. Saturación Dinámica por Límite de Potencia

Para respetar la curva de potencia constante del motor, el comando de velocidad generado por el planificador se limita dinámicamente en función de la carga izada estimada ($\hat{m}_l$). Se define una masa base M_{base} a partir de la cual el accionamiento ya no puede sostener la velocidad mecánica máxima ($v_{h,max}$) sin exceder su potencia nominal ($P_{h,nom}$):

$$v_{h,limit}(\hat{m}_l) = \begin{cases} v_{h,max} & \text{si } \hat{m}_l < M_{base} \\ \frac{P_{h,nom}}{\hat{m}_l g} & \text{si } \hat{m}_l \geq M_{base} \end{cases} \quad (52)$$

Finalmente, la señal saturada se convierte a velocidad angular, conformando la referencia maestra que ingresa al lazo regulador de velocidad:

$$v_{ref}(k) = \text{sat}(v_{req}(k), \pm v_{h,limit}(\hat{m}_l)) \quad (53)$$

$$\omega_{ref}(k) = v_{ref}(k) \left(\frac{2 i_h}{r_{hd}} \right) \quad (54)$$

2.2. Control de Traslación del Carro

Para el control del carro, se diseñó la misma arquitectura de Control Directo de Velocidad con Planificación Cinemática (Trayectoria $\rightarrow$ Velocidad $\rightarrow$ Torque).

2.2.1. Modelo del Carro para Control de Velocidad

Para el diseño del lazo interno de velocidad del carro, se adopta un modelo simplificado traslacional alrededor del punto de operación. Este modelo se obtiene a partir de la ecuación (1). En esta aproximación se desprecia la dinámica rotacional interna del accionamiento y se agrupan los acoplamientos mecánicos y perturbaciones externas en una fuerza perturbadora equivalente. Así, la dinámica horizontal del carro puede escribirse como:

$$M_t \frac{dv_t(t)}{dt} + b_t v_t(t) = F_{cmd}(t) - F_{dist}(t) \quad (55)$$

donde

$$F_{cmd}(t) = K_f T_{tm}(t) \quad (56)$$

y

$$F_{dist}(t) = \underbrace{F_{tw}(t)}_{\text{Fuerza del cable}} - \underbrace{F_{coupling}(t)}_{\text{Acoplamiento con la carga}} + \Delta_{model}(t) \quad (57)$$

siendo $T_{tm}(t)$ el torque aplicado por el motor y K_f la ganancia equivalente fuerza–torque del conjunto variador–motor–transmisión. En este caso:

$$K_f = \frac{i_t}{r_{td}} \quad (58)$$

ya que el torque en el tambor cumple $T_{td} = i_t T_{tm}$ y la fuerza lineal equivalente sobre el carro resulta $F_{cmd} = T_{td}/r_{td}$.

Aplicando la Transformada de Laplace, la función de transferencia simplificada de la planta velocidad–mando queda:

$$G_{vel,t}(s) = \frac{V_t(s)}{T_{tm}(s)} = \frac{K_f}{M_t s + b_t} \quad (59)$$

El polo mecánico resulta $P_{mec} = -b_t/M_t$. Dado que el rozamiento equivalente es pequeño frente a la inercia dominante del carro, para la sintonía se adopta la aproximación:

$$G_{vel,t}(s) \approx \frac{K_f}{M_t s} \quad (60)$$

2.2.2. Diseño del Lazo Regulador (Control PI de Velocidad)

Se utiliza un controlador PI para asegurar error nulo de velocidad ante escalones de referencia y buen rechazo de perturbaciones constantes:

$$C_{vel}(s) = K_p + \frac{K_i}{s} \quad (61)$$

La función de transferencia a lazo cerrado $T(s)$ resulta:

$$T(s) = \frac{C_{vel}(s) G_{vel,t}(s)}{1 + C_{vel}(s) G_{vel,t}(s)} \quad (62)$$

Sustituyendo $C_{vel}(s)$ y $G_{vel,t}(s) \approx \frac{K_f}{M_t s}$:

$$T(s) = \frac{\left(K_p + \frac{K_i}{s}\right) \frac{K_f}{M_t s}}{1 + \left(K_p + \frac{K_i}{s}\right) \frac{K_f}{M_t s}} \quad (63)$$

Operando algebraicamente, se obtiene:

$$T(s) = \frac{K_p K_f s + K_i K_f}{M_t s^2 + K_p K_f s + K_i K_f} \quad (64)$$

por lo que la ecuación característica de segundo orden queda:

$$M_t s^2 + K_p K_f s + K_i K_f \quad (65)$$

Se iguala con un polinomio deseado de segundo orden:

$$s^2 + 2\zeta\omega_n s + \omega_n^2 \quad (66)$$

obteniéndose las ganancias:

$$K_p = \frac{2\zeta\omega_n M_t}{K_f}, \quad K_i = \frac{\omega_n^2 M_t}{K_f} \quad (67)$$

La selección de ω_n se realiza en función del ancho de banda deseado del carro y las limitaciones de aceleración; típicamente se toma $\zeta \in [0,7, 1]$ para evitar sobrepasos.

El torque de referencia en tiempo discreto es:

$$T_{tm}^*(k) = K_p e_v(k) + K_i \sum_{j=1}^k \frac{T_s}{2} [e_v(j) + e_v(j-1)] \quad (68)$$

con $e_v(k) = v_t^*(k) - v_t(k)$ y T_s el período de muestreo.

2.2.3. Planificación Cinemática de Trayectorias

Al igual que en el izaje, en lugar de un lazo de posición tradicional que persigue una coordenada espacial, la referencia de velocidad es dictada por un algoritmo supervisor de trayectorias en el control Supervisor. Se genera una velocidad requerida (v_{req}) basada en una ley PD hacia un objetivo dinámico $x_{target}(k)$:

$$v_{req}(k) = K_p (x_{target}(k) - x_{real}(k)) + K_d v_x(k) \quad (69)$$

El valor de x_{target} es gestionado por un autómata que garantiza la evasión de colisiones antes de generar movimiento horizontal.

2.3. Diseño del Controlador de Balanceo (Anti-Sway)

Para la implementación del sistema de control de oscilación de la carga, se ha optado por una estrategia de Control Desacoplado, en lugar de un control basado en el modelo dinámico completo del sistema. A continuación, se fundamenta esta decisión y se deduce la ley de control utilizada.

2.3.1. Estructura Desacoplada

El sistema de la grúa planteado posee dos dinámicas con escalas de tiempo claramente separadas:

1. **Dinámica del Accionamiento (Lazo de Velocidad):** El control de velocidad del motor del carro tiene una respuesta rápida (tiempo de establecimiento < 100 ms).
2. **Dinámica del Péndulo:** La oscilación de la carga es un fenómeno lento, con periodos naturales en el orden de los 2 a 12 segundos (dependiendo de la altura de izaje l_h).

Bajo la hipótesis de que el lazo de velocidad está correctamente sintonizado, se asume que la velocidad real del carro v_t sigue a la referencia de velocidad v_{ref} con una dinámica despreciable:

$$v_t(t) \approx v_{ref}(t) \quad (70)$$

Esta asunción permite desacoplar la dinámica del actuador (masa del carro y fricción) de la dinámica pendular, simplificando significativamente el diseño del controlador externo.

2.3.2. Deducción de la Ley de Control No Lineal

A partir del modelo dinámico (Ley de Newton) de un péndulo simple excitado por la aceleración del carro $\ddot{x}_t$, la ecuación de movimiento está dada por:

$$\ddot{\theta}_l = -\frac{g}{l_h} \sin(\theta_l) - \frac{\ddot{x}_t}{l_h} \cos(\theta_l) \quad (71)$$

En esta arquitectura, la referencia de velocidad total enviada al variador del carro se compone de la velocidad de trayectoria planificada más la acción de control antibalaceo: $v_{ref} = v_{traj} + v_{sway}$. Se define entonces una ley de control antibalaceo proporcional al ángulo de oscilación:

$$v_{sway} = K_{sway} \theta_l \quad (72)$$

La aceleración resultante del carro impuesta sobre el pivote del péndulo será la derivada de la referencia total:

$$\ddot{x}_t \approx \dot{v}_{ref} \approx \dot{v}_{traj} + \frac{d}{dt}(K_{sway} \theta_l) = \dot{v}_{traj} + K_{sway} \dot{\theta}_l \quad (73)$$

Sustituyendo esta expresión en la ecuación no lineal (71) y agrupando los términos dependientes del estado del péndulo a la izquierda, se obtiene la ecuación diferencial del sistema en lazo cerrado:

$$\ddot{\theta}_l + \left(\frac{K_{sway}}{l_h} \cos(\theta_l) \right) \dot{\theta}_l + \frac{g}{l_h} \sin(\theta_l) = -\frac{\dot{v}_{traj}}{l_h} \cos(\theta_l) \quad (74)$$

Este resultado demuestra que la cinemática del sistema transforma la señal de velocidad inyectada en un término de amortiguamiento viscoso no lineal. Dado que en la operación normal de la grúa el ángulo de oscilación está acotado ($-\pi/2 < \theta_l < \pi/2$), el término $\cos(\theta_l)$ es estrictamente positivo. Esto garantiza que el coeficiente de amortiguamiento sea siempre mayor a cero, asegurando la extracción continua de energía cinética del péndulo y la estabilidad del sistema ante cualquier ángulo operativo, sin depender de aproximaciones de ángulos pequeños. La aceleración de la trayectoria $\dot{v}_{traj}$ actúa únicamente como una fuerza forzante acotada.

2.3.3. Sintonía por Linealización Local (Gain Scheduling)

Si bien la estabilidad está garantizada, para determinar analíticamente el valor óptimo de la ganancia K_{sway} se evalúa el comportamiento del sistema en las cercanías del punto de equilibrio estable ($\theta_l = 0, \dot{\theta}_l = 0$). El error es casi despreciable para ángulos de entre 10° y 5° , y esto permite no solo mayor eficiencia computacional sino también evitar singularidades.

Evaluando la ecuación (74) alrededor del origen ($\cos(\theta_l) \rightarrow 1$ y $\sin(\theta_l) \rightarrow \theta_l$), la dinámica local se reduce a:

$$\ddot{\theta}_l + \left(\frac{K_{sway}}{l_h} \right) \dot{\theta}_l + \left(\frac{g}{l_h} \right) \theta_l = 0 \quad (75)$$

Para garantizar un desempeño uniforme frente a la variación de la longitud del cable, se comparan los coeficientes de esta ecuación con la ecuación canónica de un sistema de segundo orden ($s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$), donde la frecuencia natural es $\omega_n = \sqrt{g/l_h}$. Igualando el término de amortiguamiento:

$$\frac{K_{sway}}{l_h} = 2\zeta\omega_n \implies K_{sway} = 2\zeta l_h \sqrt{\frac{g}{l_h}} \quad (76)$$

Simplificando, obtenemos la ganancia para un amortiguamiento crítico ($\zeta = 1$):

$$K_{sway}(l_h) = 2\sqrt{g l_h} \quad (77)$$

Dado que la altura de izaje determina directamente la dinámica del péndulo, se implementa una estrategia de *gain scheduling*, ajustando la ganancia K_{sway} dinámicamente en función de l_h para mantener las raíces del polinomio característico local invariantes durante toda la operación.

Esta ganancia teórica produce un amortiguamiento perfecto de la oscilación. Sin embargo, en la práctica esto genera correcciones horizontales notorias. Para evitar sobrecorrecciones en la posición objetivo de la carga, se debe subamortiguar el balanceo. Después de varias pruebas, se seleccionó $\zeta = 0,45$ para lograr un compromiso entre amortiguamiento efectivo y estabilidad robusta. Esto genera oscilaciones adicionales pequeñas, pero a cambio la posición de la carga se mantiene más cercana a la deseada, sin introducir retrasos excesivos.

2.4. Observadores de Estado

Se puede notar de las ecuaciones desarrolladas que la realimentación de velocidad es un parámetro crítico para garantizar la estabilidad de los lazos de velocidad. Dado que la planta física cuenta únicamente con mediciones de posición a través de encoders, la obtención de la velocidad mediante una derivada directa amplifica drásticamente el ruido de cuantificación de los sensores, desestabilizando la señal de control.

Para solucionar este problema y obtener una estimación limpia de la velocidad ($\hat{\omega}$), se implementan observadores de estado extendidos. Estos bloques combinan un modelo físico-matemático del sistema con un lazo de corrección basado en el error de estimación de posición.

El error de estimación se define como la diferencia entre la posición angular medida por el encoder y la posición estimada internamente por el observador:

$$e_\theta = \theta_{enc} - \hat{\theta}$$

Este error alimenta tres ganancias de corrección (L_1 , L_2 y L_3) que ajustan la dinámica del modelo para forzar la convergencia de los estados estimados hacia los estados reales de la planta. Las ecuaciones diferenciales que rigen la actualización de los estados en el observador están dadas por:

$$\begin{aligned}\dot{\hat{\theta}} &= \hat{\omega} + L_1 e_\theta \\ \dot{\hat{\omega}} &= \frac{1}{J_{eq}} \left(T_{ref} - b_{eq} \hat{\omega} + L_3 \int e_\theta dt \right) + L_2 e_\theta\end{aligned}$$

Donde:

- J_{eq} y b_{eq} representan la inercia y fricción viscosa equivalentes reflejadas al eje del motor.
- T_{ref} es el torque de referencia comandado por el lazo de velocidad interno.
- $L_3 \int e_\theta dt$ actúa como un estimador de perturbaciones, acumulando el error para compensar dinámicas no modeladas o variaciones de carga.

La arquitectura del observador es idéntica tanto para el mecanismo del carro como para el de izaje. Sin embargo, en el eje de izaje se introduce un término adicional de pre-compensación $T_{grav} = M_s g \frac{r_{hd}}{2i_h}$ para contrarrestar el peso en vacío del spreader (M_s), evitando así un error de estimación transitorio en el instante de arranque.

Finalmente, la velocidad lineal observada ($\hat{v}$) que retroalimenta al controlador regulatorio principal se obtiene mediante la relación de transmisión cinemática de cada mecanismo respectivo:

$$\hat{v} = \hat{\omega} \cdot k_{out}$$

Siendo la constante de conversión $k_{out} = \frac{r_{td}}{i_t}$ para el sistema del carro y $k_{out} = \frac{r_{hd}}{2i_h}$ para el sistema de izaje.

2.4.1. Cálculo de Ganancias

Para determinar los valores empíricos de las ganancias de corrección vectoriales $L = [L_1, L_2, L_3]^T$, se empleó la técnica de asignación de polos (Pole Placement) sobre un modelo de la planta en el espacio de estados aumentado.

El sistema mecánico del accionamiento se modeló definiendo como variables de estado la posición $x_1 = \theta$, la velocidad $x_2 = \omega$ y un estado extendido $x_3 = T_{dist}$, el cual representa el torque de perturbación total no modelado (acción integral). El sistema continuo aumentado resulta:

$$\dot{x} = A_{aug} x + B_{aug} u$$

$$y = C_{aug} x$$

Donde las matrices dinámicas del sistema son:

$$A_{aug} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & -\frac{b_{eq}}{J_{eq}} & \frac{1}{J_{eq}} \\ 0 & 0 & 0 \end{bmatrix}, \quad C_{aug} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$$

La sintonía de este observador se rige por el principio de separación de anchos de banda. Para garantizar que la dinámica de estimación sea lo suficientemente veloz y no introduzca retardos de fase en el lazo de control regulatorio, la dinámica del observador se diseñó para ser significativamente más rápida que la dinámica de la planta. Específicamente, el polo base del observador (ω_{obs}) se ubicó de manera que su respuesta sea 20 veces más rápida que el polo dominante de la planta a lazo abierto ($p_{planta} = -b_{eq}/J_{eq}$).

Finalmente, mediante la función *place*, se calculó el vector de ganancias L que fuerza a la matriz característica del error del observador a poseer los autovalores deseados.

Cabe destacar una consideración numérica importante en la implementación computacional: la función *place* presenta inestabilidades e inconvenientes para invertir matrices cuando se exige la asignación de autovalores repetidos exactos. Para evitar este comportamiento numérico sin alterar la dinámica real del estimador, los tres polos del observador se separaron levemente entre sí (con variaciones del 1 % al 2 % respecto a la frecuencia base ω_{obs}). Esta técnica asegura un cálculo robusto de las ganancias y garantiza una convergencia asintótica de las estimaciones $\hat{\theta}$, $\hat{\omega}$ y del torque de perturbación $\hat{T}_{dist}$.

2.5. Modelo del Controlador Regulatorio en Simulink

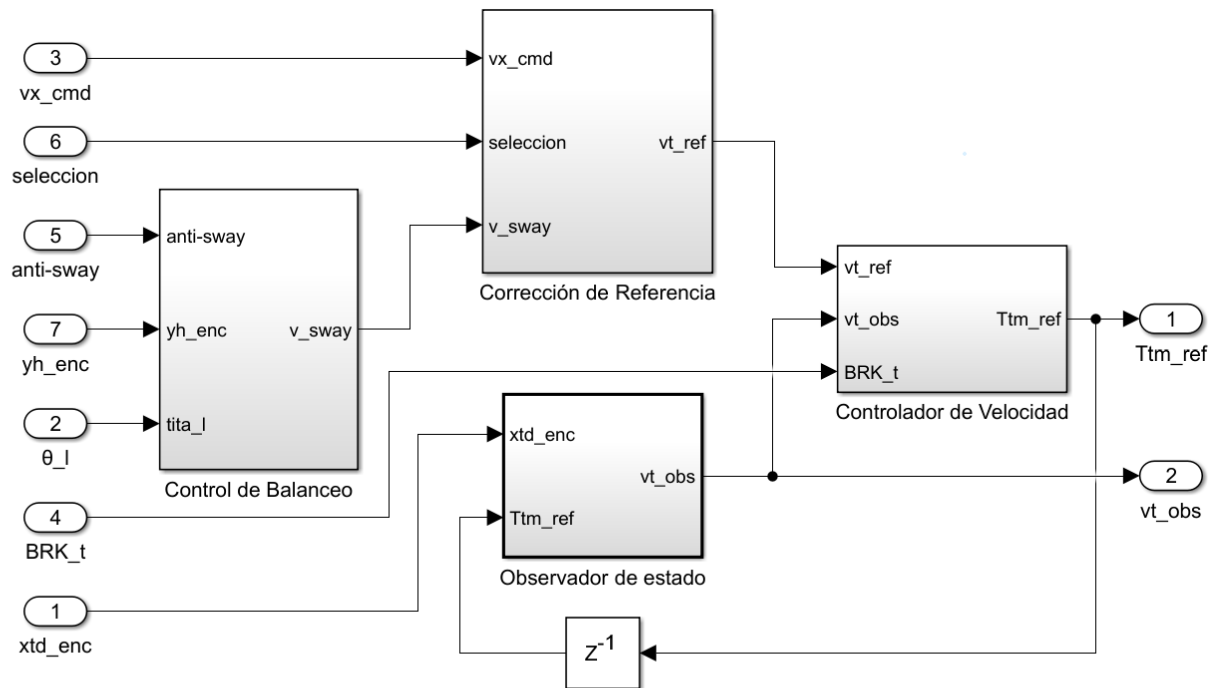


Figura 7: Control Regulatorio del Carro (Trolley).

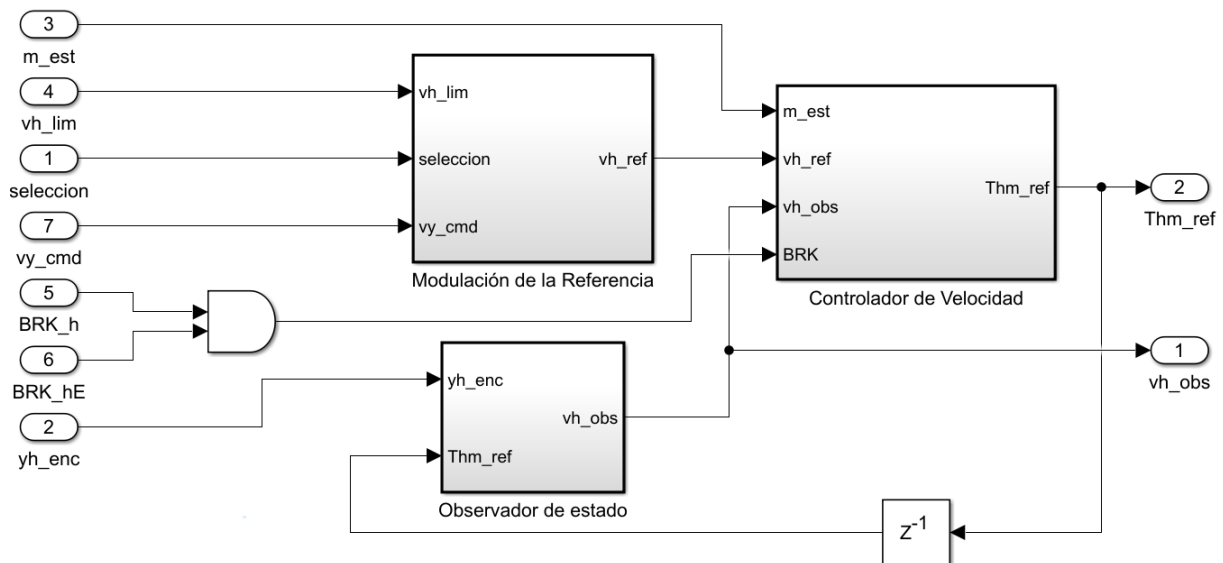


Figura 8: Control Regulatorio del Izaje (Hoist).

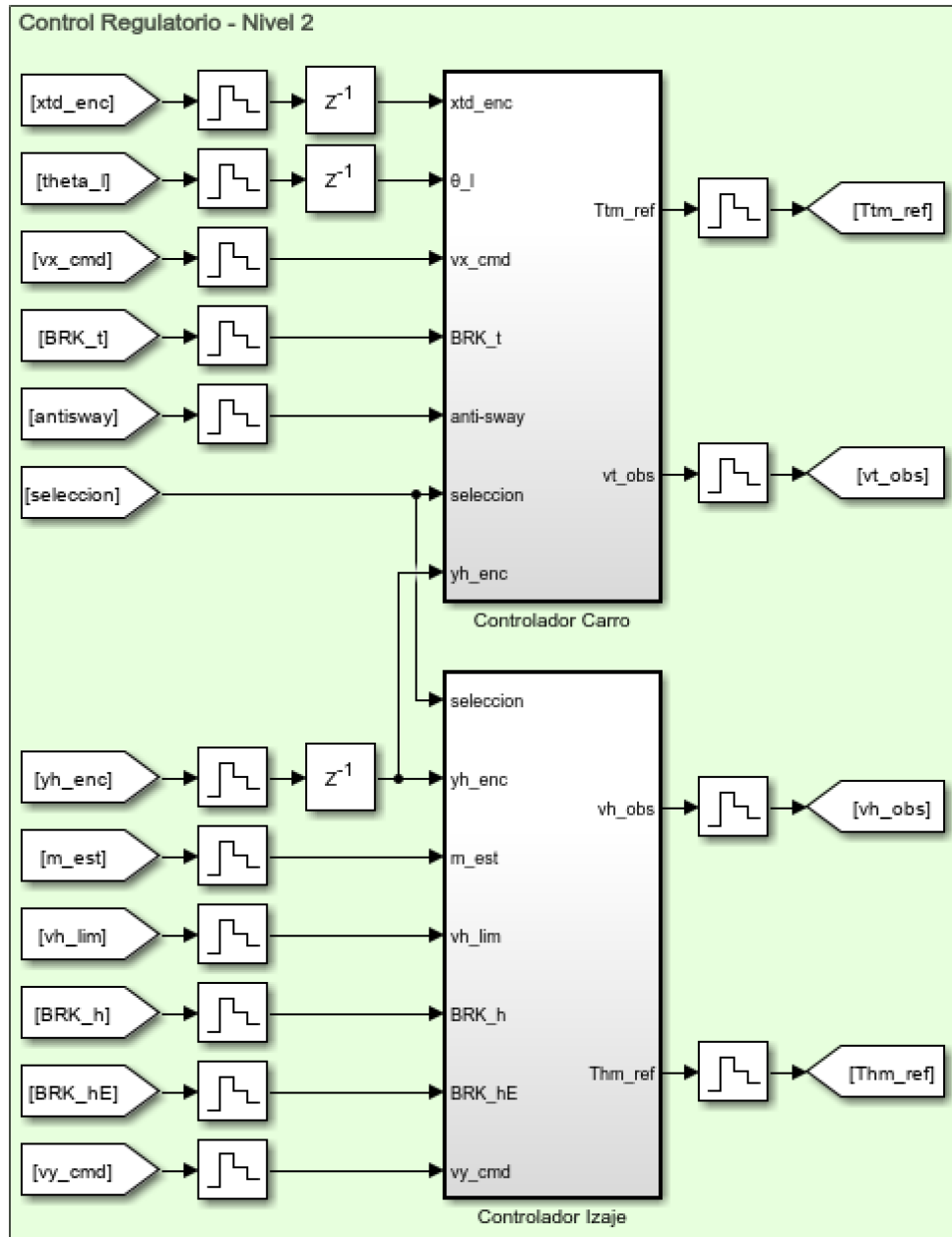


Figura 9: Modelo del Control Regulatorio en Simulink.

3. Diseño del Control Supervisor (Nivel 1)

El control supervisor actúa como el cerebro operativo de la grúa. Ejecutado a un tiempo de muestreo $T_{s1} = 20$ ms, su objetivo es tomar decisiones lógicas, garantizar la seguridad y coordinar los comandos que se envían al control regulatorio de bajo nivel.

Para organizar esta lógica, la arquitectura se dividió en múltiples procesos paralelos, permitiendo que el control evalúe simultáneamente el modo de operación, la seguridad y el entorno.

3.1. Autómata Principal de Modos de Operación

Como primer bloque se desarrolla una máquina de estados. Este es el hilo de ejecución primario que dicta el comportamiento global de la grúa. Su objetivo es gestionar de manera segura las transiciones entre los distintos modos de operación:

- **Stand By:** Estado de reposo y espera de comandos. Es el estado inicial del sistema, donde la grúa permanece con los frenos cerrados. Este estado también se activa automáticamente después de un período de inactividad o tras una operación completa, garantizando que la grúa no quede en una posición vulnerable.
- **Modo Manual:** Permite al operador tomar el control directo de las velocidades de los ejes a través del joystick. En conjunto con el bloque *Manejo Cruceta* se encarga de traducir las señales del joystick en consignas de velocidad para el control regulatorio, permitiendo movimientos precisos y suaves. Este estado es activado siempre que se detecte una consigna en el joystick.
- **Modo Automático:** Es el estado más complejo, ya que debe controlar la grúa sin señales del operador. Incluye múltiples estados internos que permiten seleccionar el contenedor a mover. Luego toma el control de la grúa para mover el mismo desde un punto A hasta un punto B. A través de dos funciones Matlab, genera una curva de movimiento segura, moviendo el spreader a estados intermedios que salen como consignas de velocidad al control regulatorio. Estas funciones manejan dos casos distintos: uno cuando no se conoce el perfil de obstáculos, el cual genera una trayectoria segura, y otro cuando sí se conoce, utilizando el mapa de alturas generado por el bloque paralelo de *Índices y Perfil*.
- **Modos de Error y Recuperación:** En caso de detectarse una falla (propia del control supervisor o del control de seguridad), el autómatas aborta cualquier operación actual y fuerza al sistema a adoptar un estado de resguardo para proteger la integridad de la carga y el equipo. Esto lo hace activando los frenos según corresponda. La recuperación se puede activar con el botón de reset, limitando la velocidad y dirección de movimiento manual. Una vez que el operador termina la maniobra de recuperación, puede volver al estado Stand By presionando el botón de reset nuevamente.

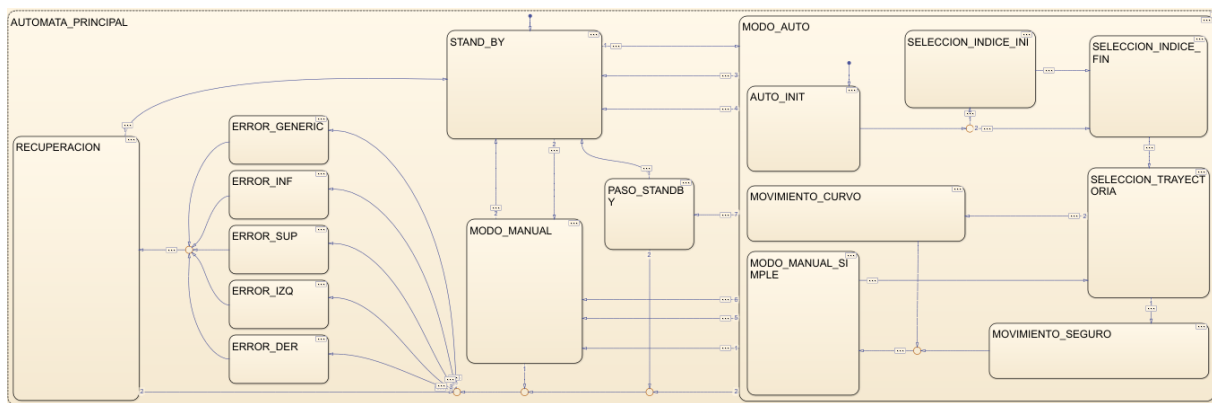


Figura 10: Máquina de Estados en Stateflow.

3.2. Gestión de Twistlocks

Este bloque paralelo se encarga exclusivamente de la lógica del spreader. Administra los tiempos y las confirmaciones de los sensores para tomar (**CERRANDO_TLK**) o soltar (**ABRIENDO_TLK**) el contenedor, asegurando que la grúa nunca intente un movimiento de izaje si el anclaje no es mecánicamente seguro. Posee 4 estados internos: **VACIO**, estado inicial que mueve el spreader a la velocidad máxima permitida, **CERRANDO_TLK**, que se activa cuando la tensión en el cable disminuye lo suficiente para indicar que el spreader se apoyó en un contenedor, **CARGADO**, que estima la masa del contenedor, limita la velocidad según la curva de potencia y detecta sobrecarga, y **ABRIENDO_TLK**, que análogamente al estado de cerrar, detecta que la tensión disminuya para abrir los twistlocks. Finalmente se transiciona al estado inicial una vez superada cierta altura de

seguridad.

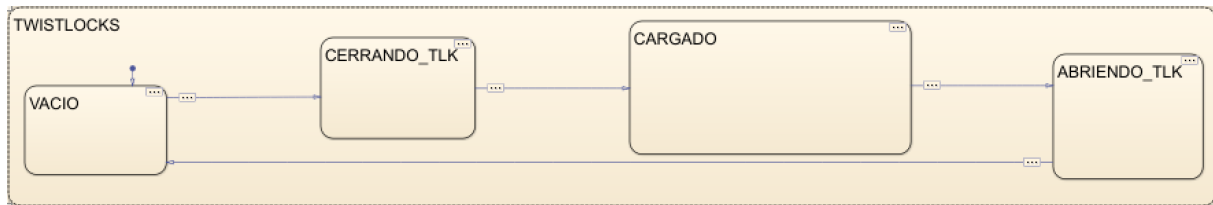


Figura 11: Lógica Twistlocks en Stateflow.

3.3. Suavizador de Consignas de Velocidad

Se encarga de aplicar filtros de suavizado a las consignas de velocidad provenientes del autómata principal. Esto es especialmente importante en el modo manual, donde las señales del joystick pueden ser abruptas. El bloque implementa límites de jerk, aceleración y velocidad, garantizando que cualquier cambio en la referencia se realice de manera gradual, evitando movimientos bruscos que puedan comprometer la estabilidad o seguridad de la operación.

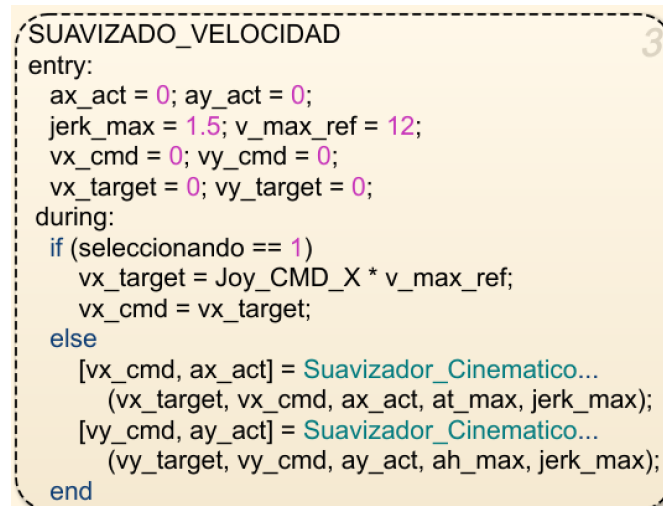


Figura 12: Suavizador de Velocidad.

3.4. Análisis Espacial, Perfil de Obstáculos y Actualización de Cruceta

Para que el modo automático pueda trazar rutas seguras, la grúa debe conocer su entorno. Este bloque procesa los datos de los sensores de escaneo del LIDAR y actualiza continuamente un mapa de alturas o perfil de obstáculos bajo el área de trabajo. Esto garantiza que cualquier trayectoria generada esquivе las pilas de contenedores existentes. Al mismo tiempo, calcula los índices en los que se encuentra el carro. El perfil de alturas se representa como una envolvente de seguridad, con márgenes laterales y superiores que garantizan que la grúa no intente pasar por zonas donde pueda colisionar con obstáculos. Por último, también se encarga de actualizar la posición de la cruceta, que es el punto de referencia para el operador, asegurando que siempre se muestre la posición real del spreader en la interfaz.

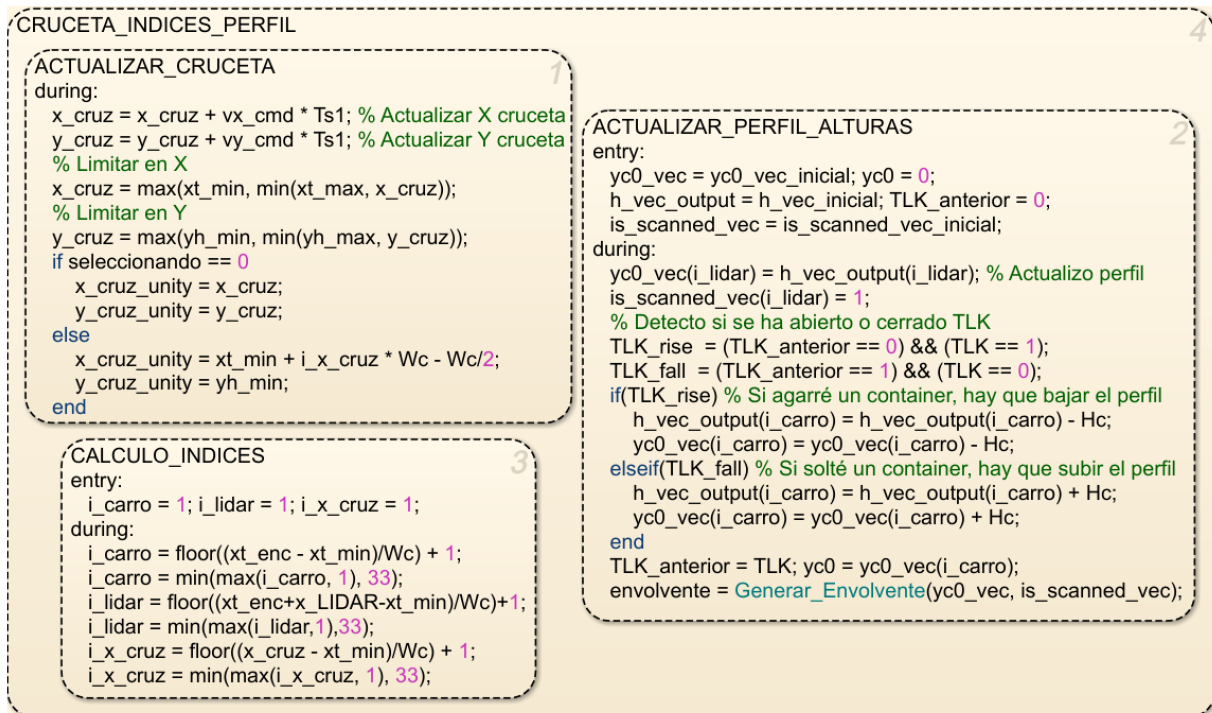


Figura 13: Índices y Perfil de Obstáculos en Stateflow.

3.5. Supervisión de Comunicación (Watchdog Heartbeat)

Este bloque constituye un anillo de seguridad vital. Se encarga de emitir pulsos de vida (*heart-beats*) a intervalos regulares. En el control de seguridad, otro bloque observa esta señal. Si se detecta una falta de pulsos (superando un tiempo preestablecido), el sistema de seguridad asume una pérdida de comunicación, interrumpiendo cualquier movimiento en curso y enviando al sistema al estado de error.

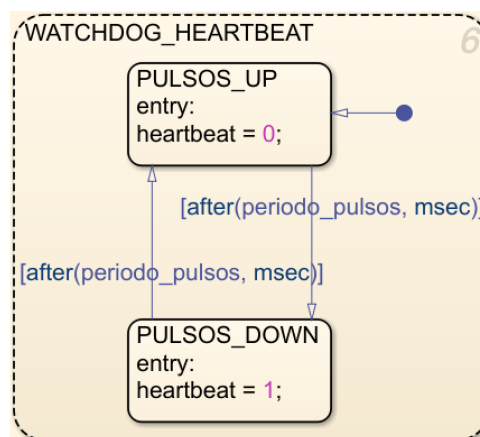


Figura 14: Watchdog Heartbeat en Stateflow.

3.6. Límites Suaves

Este bloque de código actúa como una barrera de seguridad preventiva. Antes de que el carro o el izaje alcancen los límites físicos extremos de su carrera (donde actuarían los finales de carrera), este bloque recorta gradualmente las referencias de velocidad, forzando una detención suave y controlada dentro del área de trabajo válida.

3.7. Modelo del Control Supervisor en Simulink

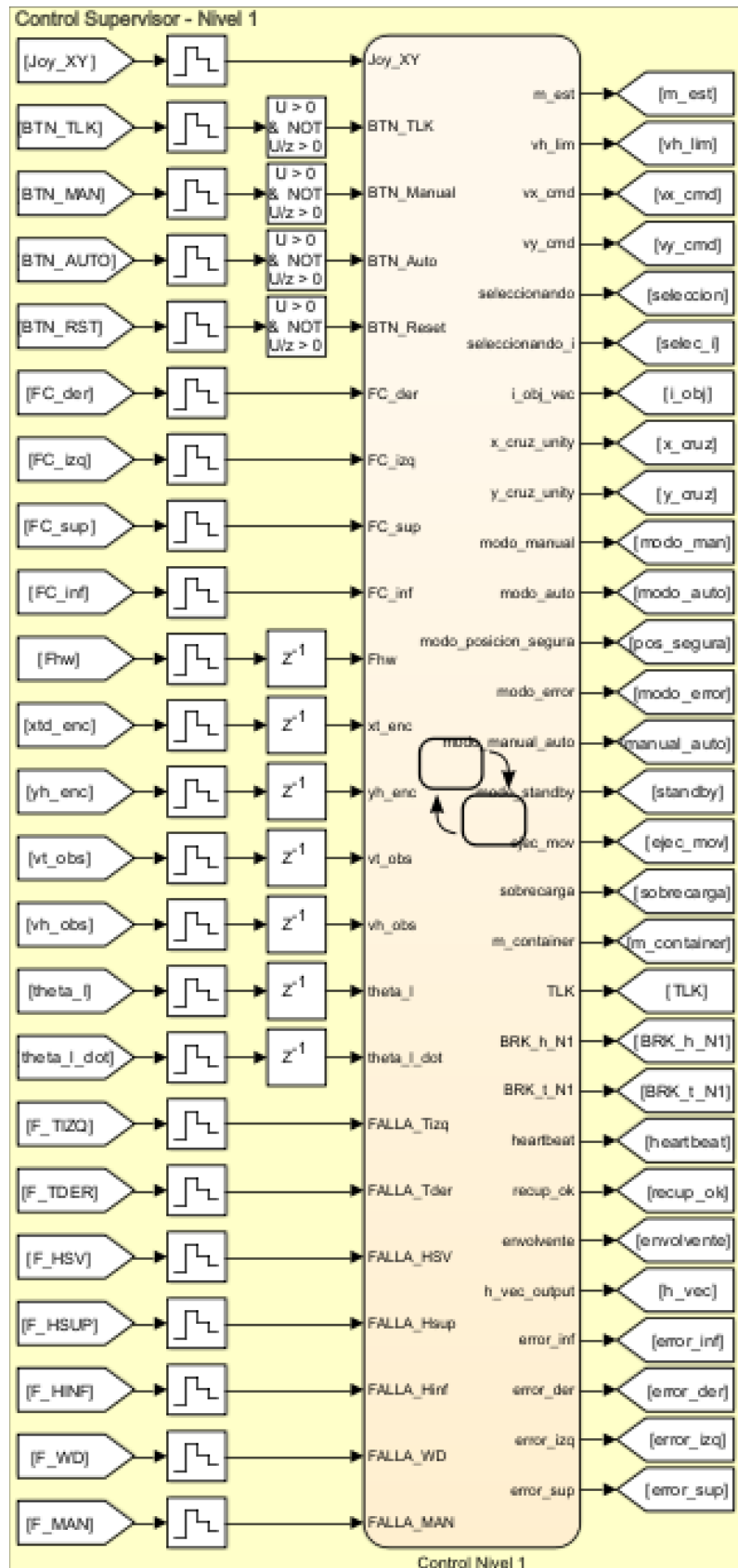


Figura 15: Modelo del Control Supervisor en Simulink.

3.8. Control del Joystick y Botones Físicos

Para controlar la grúa, se utiliza un joystick físico conectado a la estación de operación. Este dispositivo permite al operador controlar directamente las velocidades de traslación e izaje, así como activar comandos específicos (como el cierre de twistlocks). El bloque de manejo de entradas en el control supervisor procesa estas señales para generar referencias adecuadas para el control regulatorio.

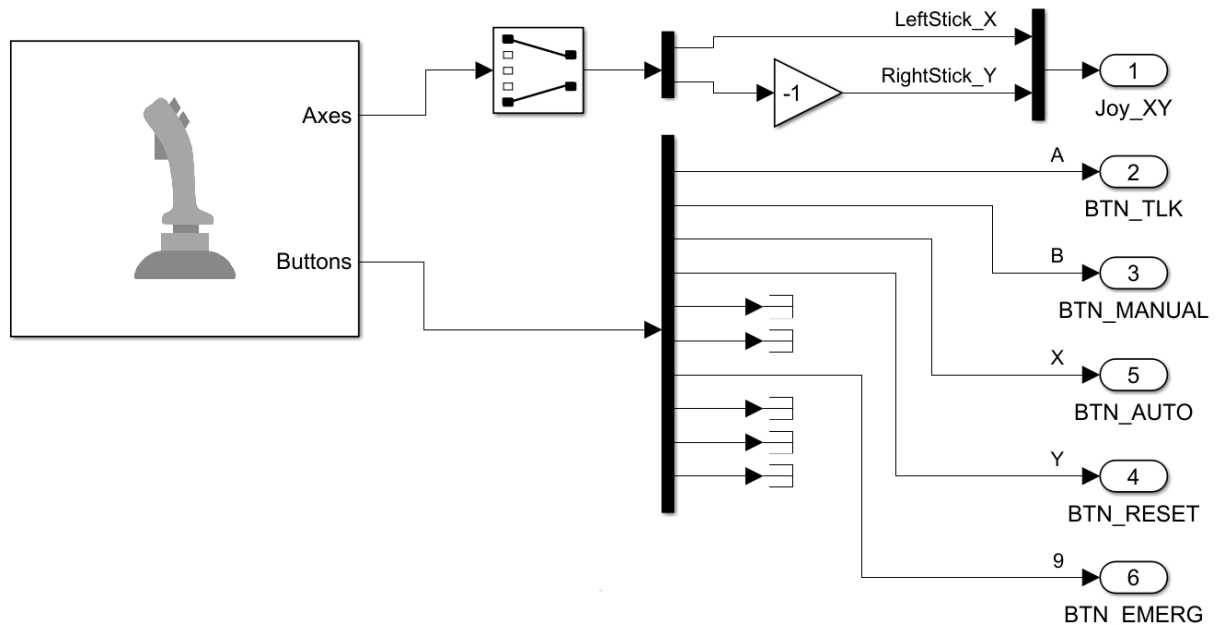


Figura 16: Bloque de Configuración del Joystick en Stateflow.

4. Diseño del Control de Seguridad (Nivel 0)

El Nivel 0 representa la capa de protección más baja y con mayor jerarquía dentro de la arquitectura de control. Su función exclusiva es salvaguardar la integridad de la grúa, la carga y el personal. Este nivel tiene la autoridad para sobrescribir cualquier comando proveniente del Nivel 1, cortando la energía de los actuadores y aplicando los frenos mecánicos ante cualquier anomalía.

La lógica de seguridad se modeló mediante dos procesos paralelos continuos:

4.1. Autómata de Estados de Seguridad

Esta máquina de estados supervisa constantemente el estado físico de la máquina a través de la redundancia de sensores. Durante la operación normal, el sistema permanece en el estado **SISTEMA_OK**.

Dentro de este estado nominal, el bloque realiza un cálculo dinámico crítico: ajusta en tiempo real el límite de velocidad de izaje permitido (vh_{lim}) en función de la masa actual de la carga. Esto garantiza que el motor nunca exceda su potencia nominal (P_{nom}), comportándose según una curva de potencia constante (sobrevelocidad).

El sistema evaluará continuamente condiciones de falla para transicionar a estados de emergencia específicos, en los cuales se aplican inmediatamente los frenos de emergencia ($BRK_t = 0$, $BRK_{hE} = 0$), según corresponda:

- **Emergencia de Izaje:** Se activa uno de los 3 estados según corresponda: si la velocidad observada supera en un 15 % al límite dinámico calculado (sobrevelocidad), si se activan los finales de carrera superior/inferior (FCE_sup, FCE_inf).
- **Emergencia de Carro:** Se activa uno de los 2 estados: si el carro activa los finales de carrera extremos (FCE_izq, FCE_der).
- **Emergencia Manual:** Detención forzada accionada por el operador mediante el botón físico de emergencia (BTN_Emerg).
- **Emergencia de Comunicación (EMERGENCIA_WD):** Disparada por la pérdida de comunicación con el control supervisor, utilizando el watchdog.

Una vez mitigada la causa de la parada, el sistema exige un rearme manual (BTN_Reset) que hace transicionar a la máquina a un MODO_RECUPERACION antes de permitir volver a operación nominal.

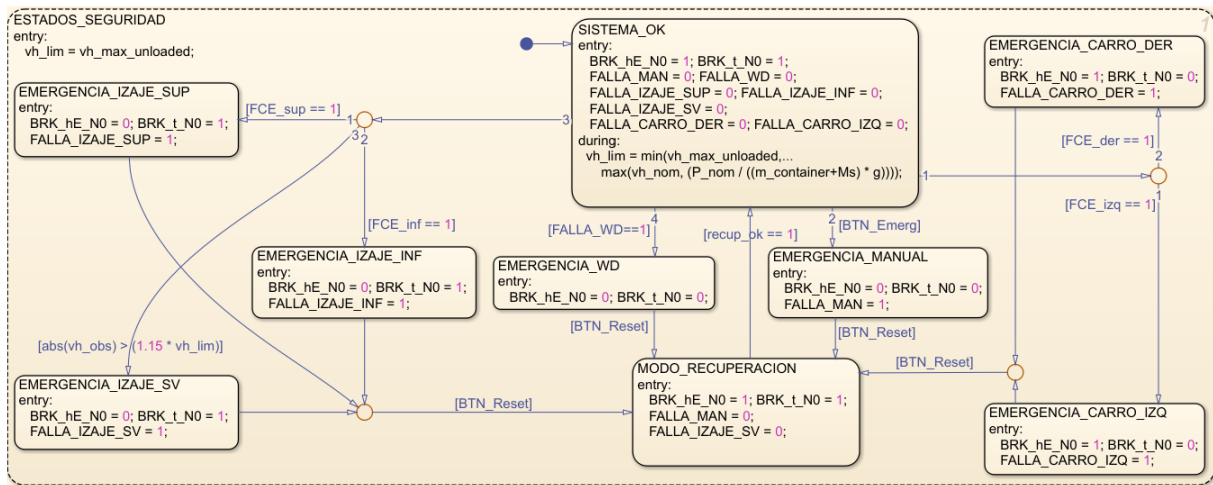


Figura 17: Estados de Seguridad en Stateflow.

4.2. Monitor de Comunicaciones

Este segundo proceso paralelo funciona como un supervisor de la integridad de la red de datos. El control supervisor debe enviar una señal de latido (*heartbeat*) que alterna su estado constantemente.

La máquina de estados del Watchdog alterna entre HEARTBEAT_DOWN y HEARTBEAT_UP persiguiendo esta señal. Si la señal se congela y el sistema permanece en un solo estado por una cantidad de ciclos de reloj que supere el límite seguro (*counter_limit*), se declara una falla de red (FALLA_WD), lo que dispara la parada de emergencia en el proceso principal para evitar que la grúa continúe moviéndose con el último comando recibido.

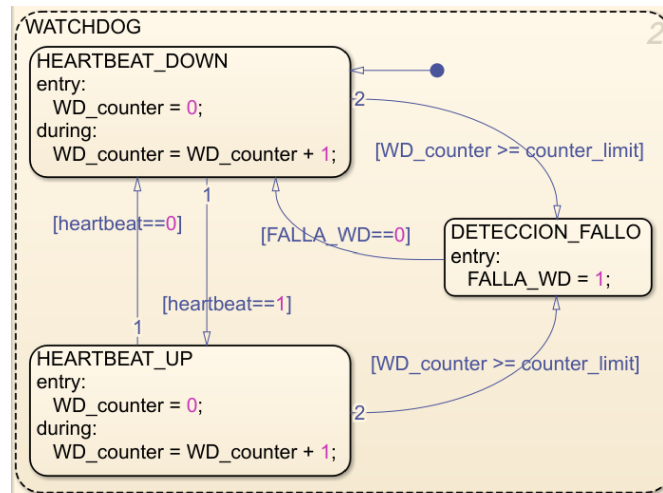


Figura 18: Watchdog en Stateflow.

4.3. Modelo del Control de Seguridad en Simulink

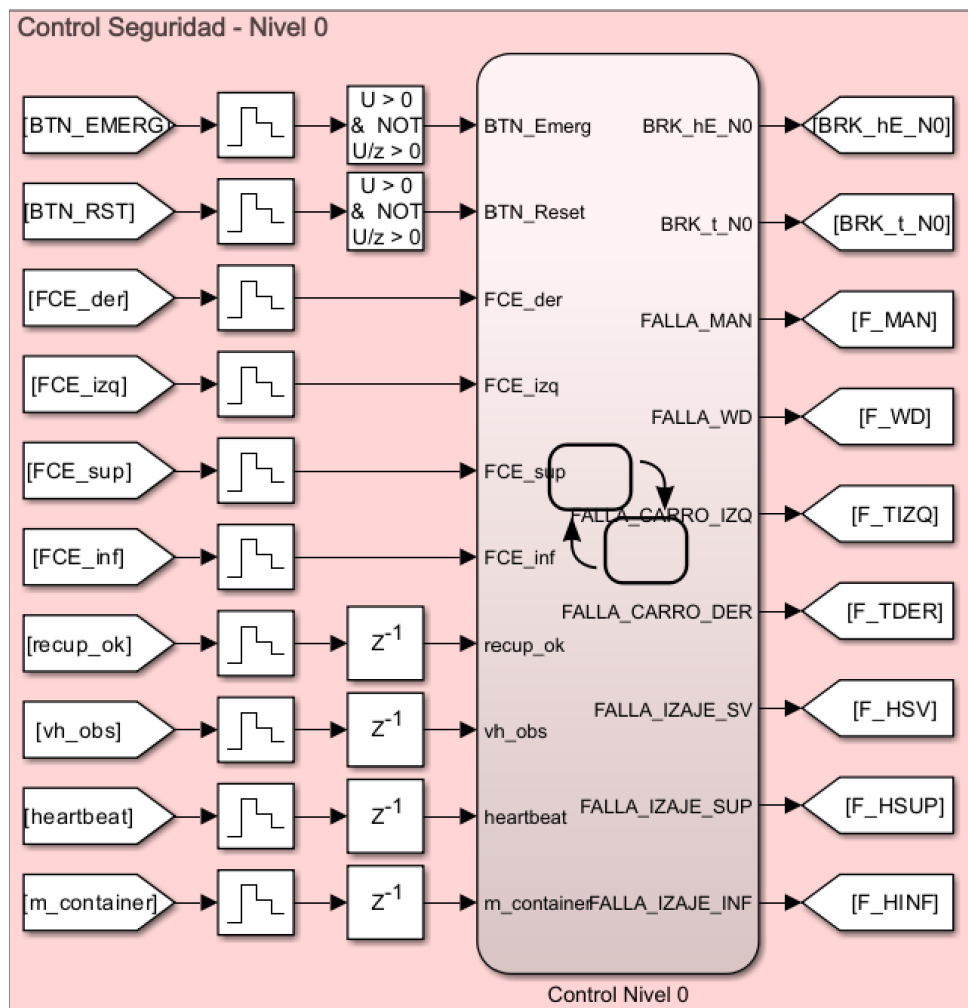


Figura 19: Modelo del Control de Seguridad en Simulink.

5. Comunicación y Animación con Unity

Para la visualización en tiempo real de la grúa y su entorno, se implementó un bloque de comunicación que transmite los estados críticos del sistema a una aplicación desarrollada en Unity. Esta aplicación renderiza una representación 2D de la grúa, los contenedores y el barco, permitiendo al operador monitorear visualmente la operación.

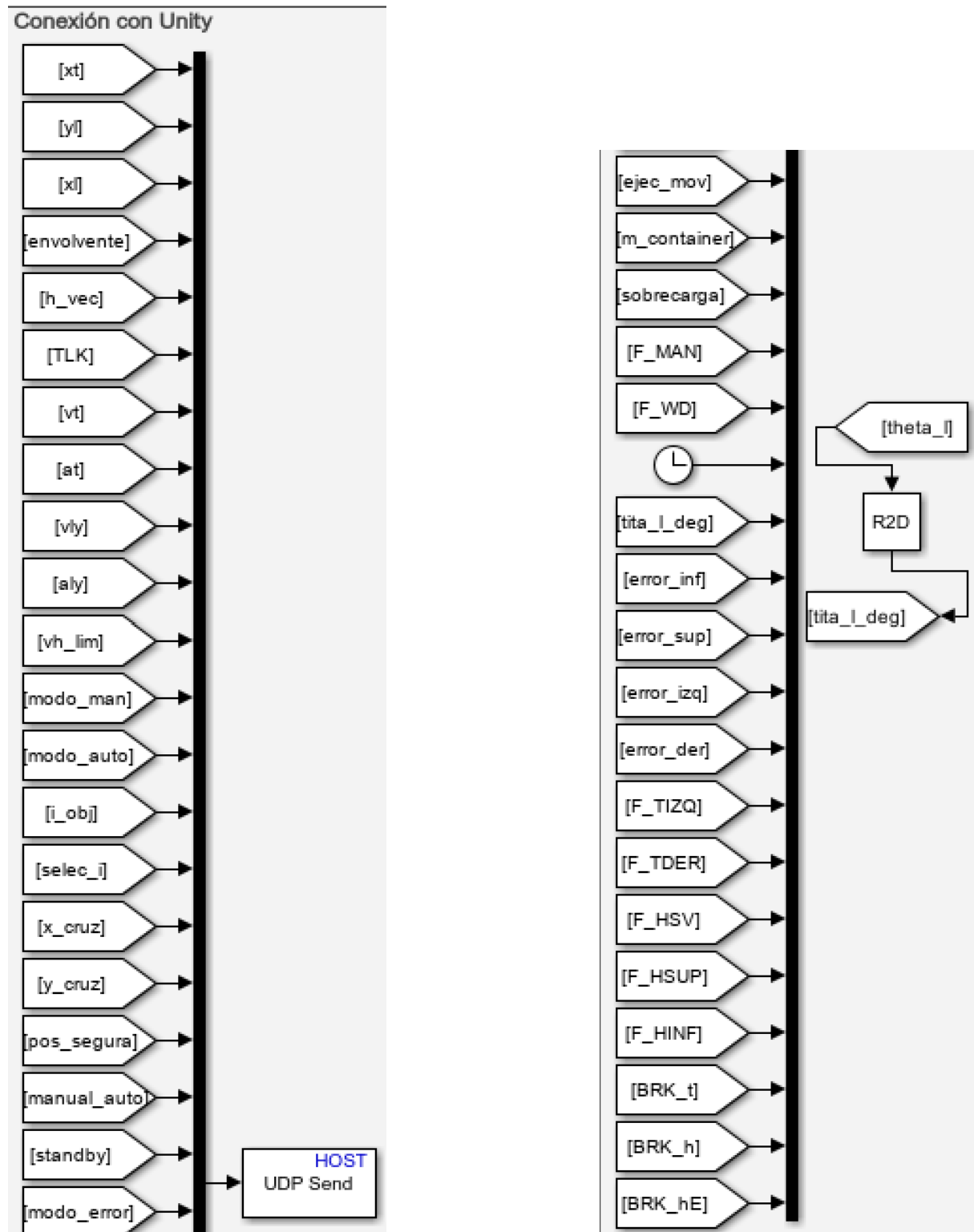


Figura 20: Bloque de Comunicación con Unity en Simulink.

A continuación se presentan algunas capturas en diferentes estados de la grúa, mostrando la posición del spreader, la carga y el entorno. Estas visualizaciones son fundamentales para que el operador tenga una referencia clara de la operación en curso, especialmente en modo automático.

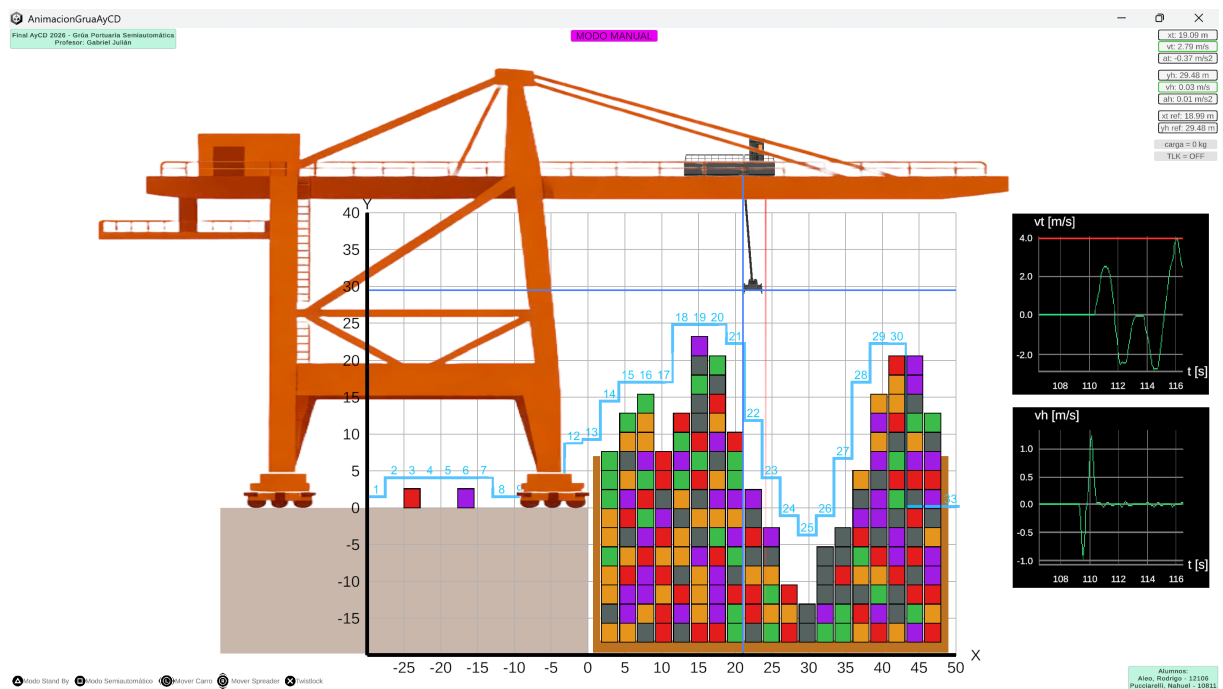


Figura 21: Modo Manual. Nótese la envolvente de seguridad del perfil de obstáculos y la zona aún sin medir con el LIDAR (índice 31 en adelante).

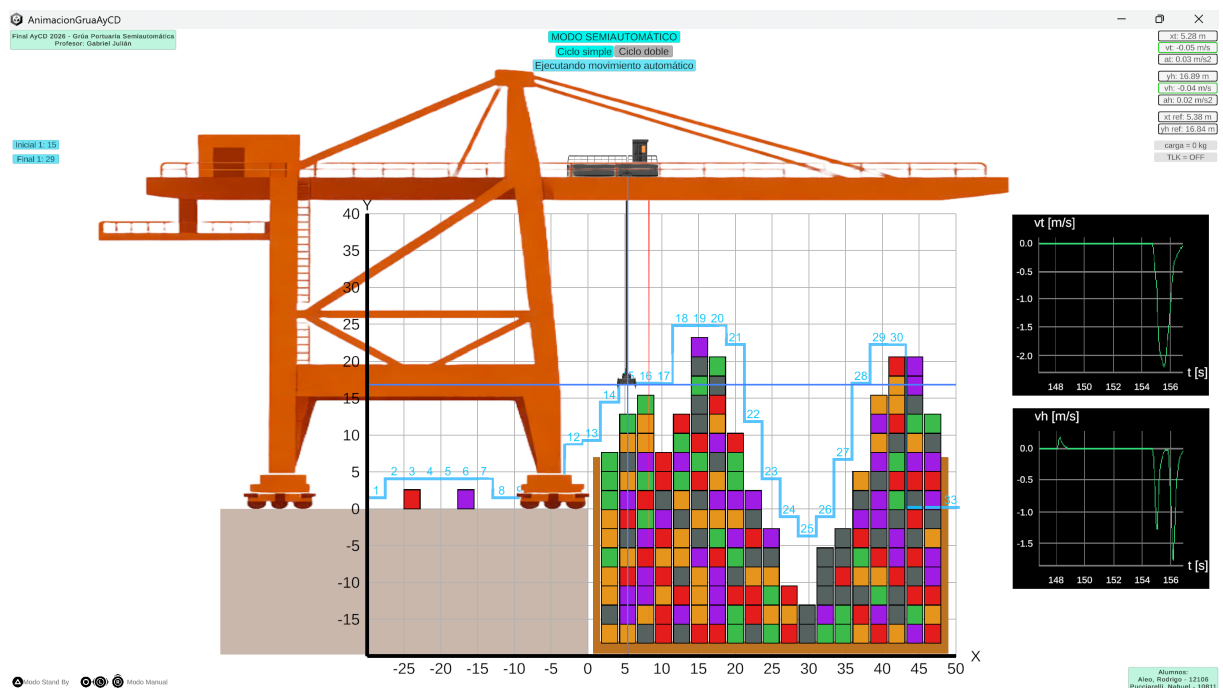


Figura 22: Modo Automático.

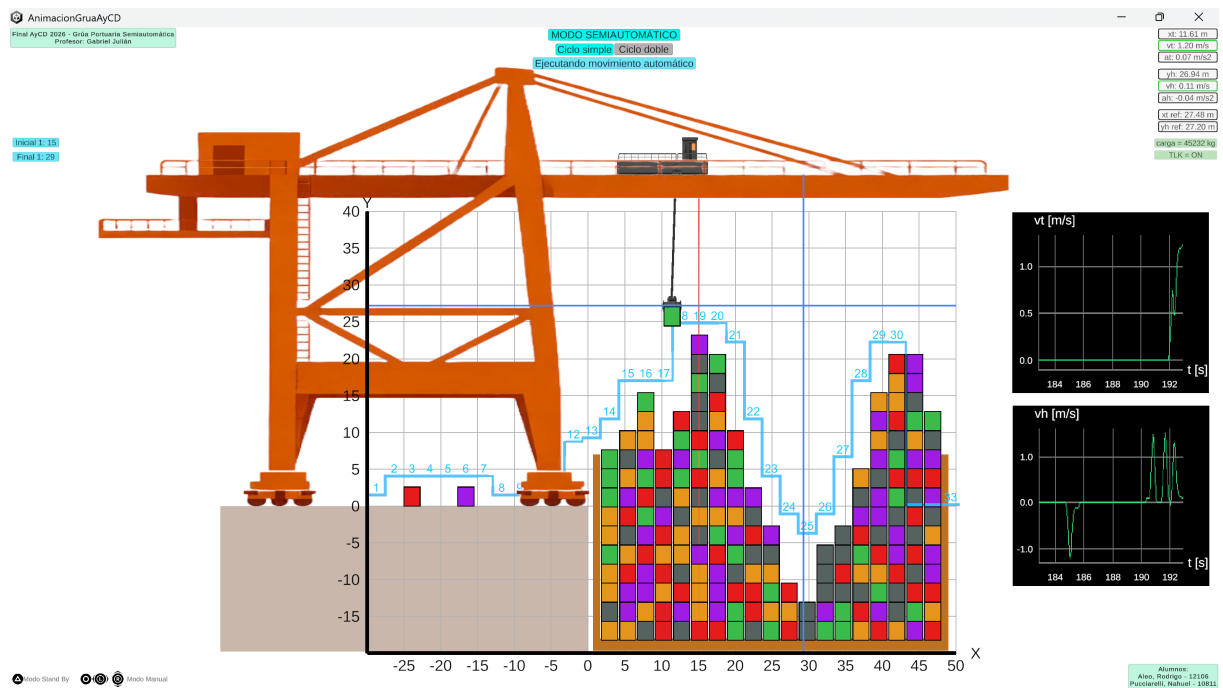


Figura 23: Modo Automático. Nótese la cruceta que muestra la posición objetivo a llevar el contenedor, con la masa ya estimada y la velocidad limitada según la curva de potencia.

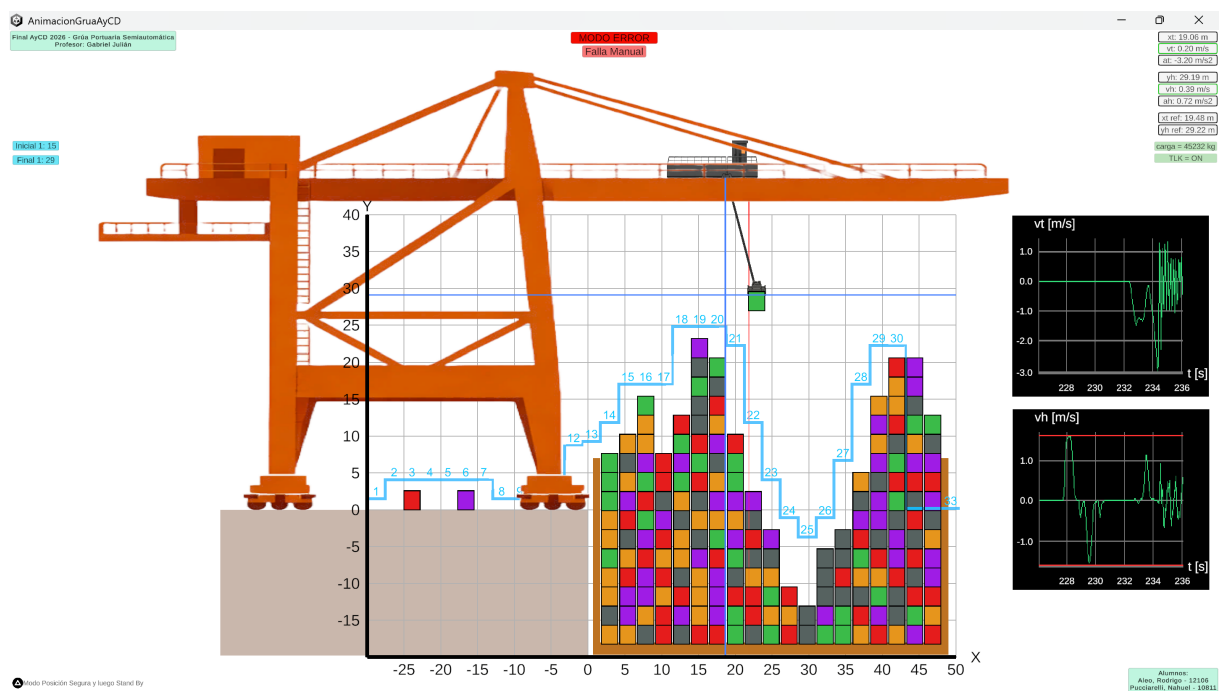


Figura 24: Modo Error Manual. Nótese que la grúa se detiene y se muestra un mensaje de error por haber apretado el botón de emergencia

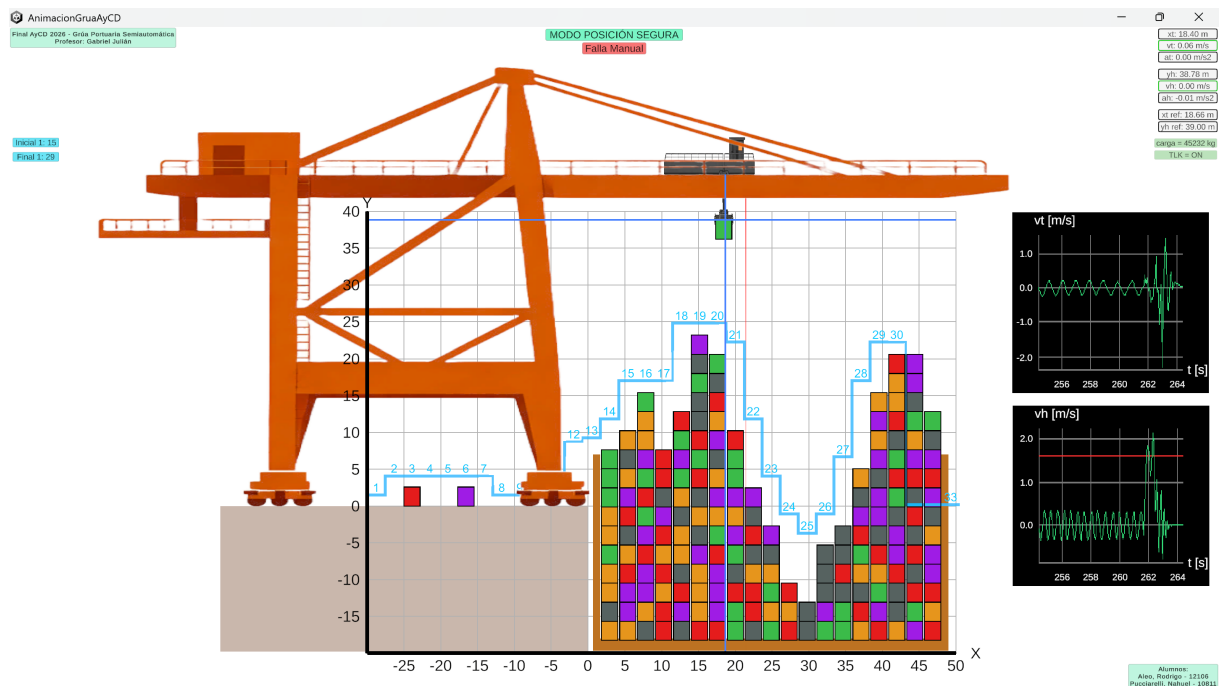


Figura 25: Modo de Posición Segura. De forma automática, la grúa se mueve a una posición de resguardo para proteger la integridad de la carga y el equipo.

6. CODESYS y ST

Todos los controladores se han diseñado en Stateflow, sin embargo, para la implementación en hardware real, se debe traducir la lógica desarrollada a un programa en CODESYS utilizando el lenguaje de programación estructurado (ST). Este proceso implica mapear cada bloque funcional a funciones y estructuras de control equivalentes en ST, asegurando que la lógica de control se mantenga intacta durante la transición al entorno de ejecución real. Puede verse a continuación la estructura resultante:

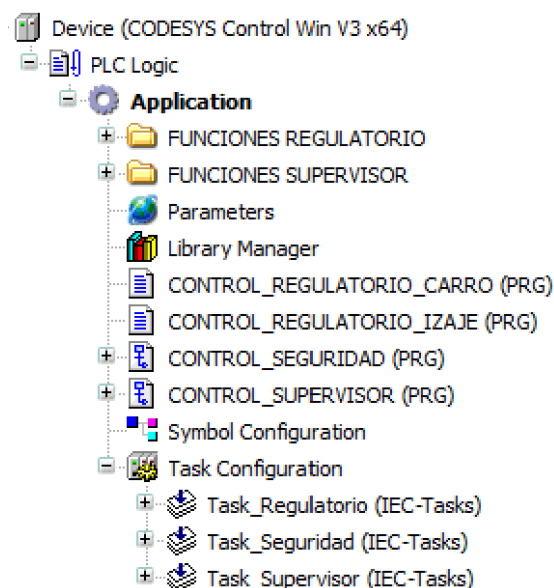


Figura 26: CODESYS - Árbol de componentes.

Los 4 POU's principales corresponden a los 3 niveles de control, separado el control regulatorio en control del carro y del izaje. Estos 4 programas se ejecutan de forma cíclica y en paralelo. Sin embargo en CODESYS debido a la utilización de un PLC virtual con una sola CPU, estas tareas se ejecutan de forma secuencial pero de manera intercalada. A pesar de la limitación, se ejecuta a tal velocidad que, a fines prácticos, el comportamiento es idéntico al de un sistema con múltiples CPUs ejecutando tareas en paralelo (o múltiples PLCs).

6.1. Control Regulatorio (Nivel 2) en CODESYS

Remitiéndose a la sección de diseño del control regulatorio, se rediseñó usando la misma lógica pero representándola con bloques de función de Matlab:

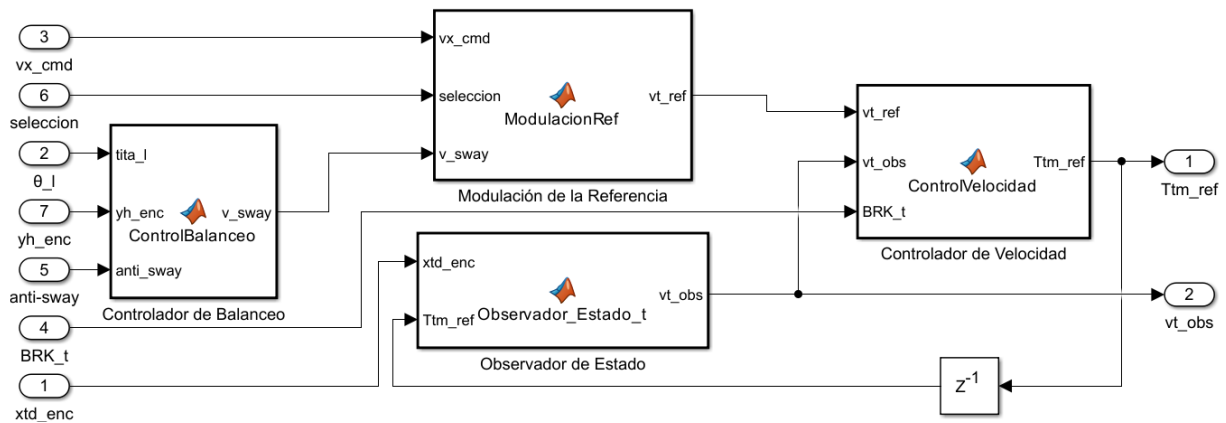


Figura 27: Control del Carro con funciones de Matlab.

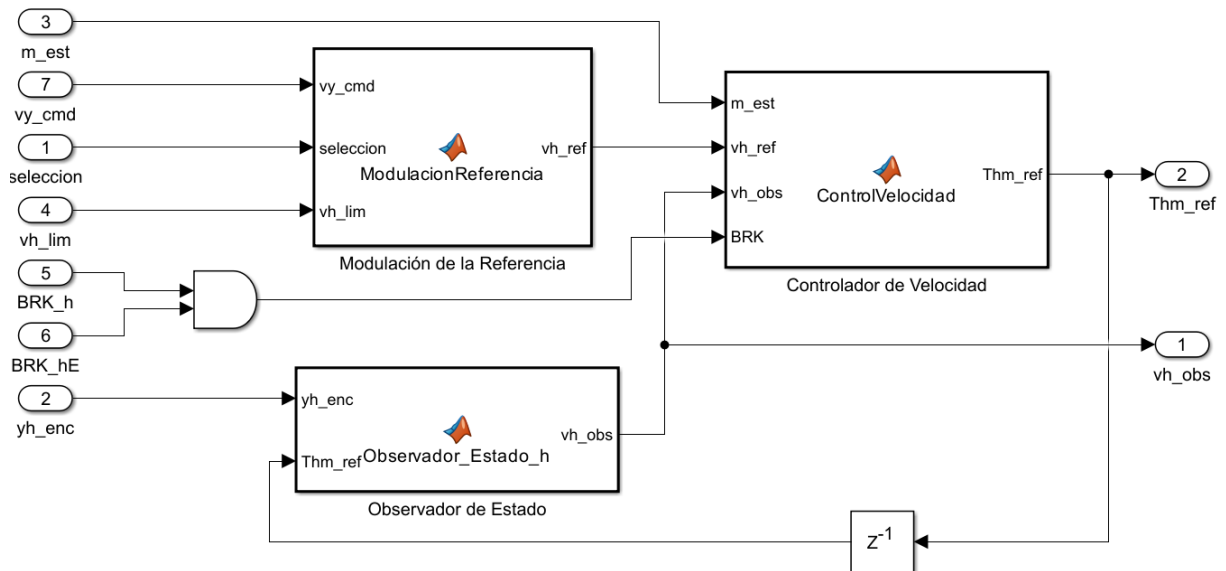


Figura 28: Control de Izaje con funciones de Matlab.

Una vez llevado a bloques de función, es relativamente directo realizar la traducción a ST, manteniendo la misma lógica de control.

```

PROGRAM CONTROL_REGULATORIO_IZAJE
VAR_INPUT
    yh_enc      : LREAL;    // Posición del encoder de izaje
    yh_ref      : LREAL;    // Referencia de posición objetivo
    vy_cmd      : LREAL;    // Comando de velocidad manual
    vh_lim      : LREAL;    // Límite de velocidad dinámico (calculado según m_est)
    m_est       : LREAL;    // Masa estimada total (spreader + carga)

    BRK_h       : INT;      // Freno de operación (1 = abierto)
    BRK_hE      : INT;      // Freno de emergencia (1 = abierto)

    modo_manual : BOOL;     // Modo de operación (TRUE = Manual)
END_VAR

VAR_OUTPUT
    Thm_ref     : LREAL;    // Comando de torque al motor de izaje
    vh_obs      : LREAL;    // Velocidad observada del izaje
END_VAR

// PASO 1: Observador de Estado
inst_Observador(
    pos_enc      := yh_enc,
    T_ref_in     := Thm_ref,
    k_conv_in    := (2.0 * ih / rhd),
    k_conv_out   := (rhd / (2.0 * ih)),
    T_grav       := (Ms * g * rhd / (2.0 * ih)), // Corrección observador
    L1           := Lh_1,
    L2           := Lh_2,
    L3           := Lh_3,
    b_eq         := bh_eq,
    J_eq         := Jh_eq,
    Ts           := Ts2,
    theta_0      := titahm_0,
    w_0          := whm_0,
    T_0          := Thm_0,
    v_obs        => vh_obs
);

// PASO 2: Generador de Trayectoria (Curva S)

```

Figura 29: Control de Izaje en CODESYS (ST).

6.2. Control Supervisor (Nivel 1) en CODESYS

Para el control supervisor se debe repensar la forma de traducir los bloques de Stateflow a SFC, ya que no es tan directo. De este proceso se obtuvo un conjunto de varios bloques en paralelo:

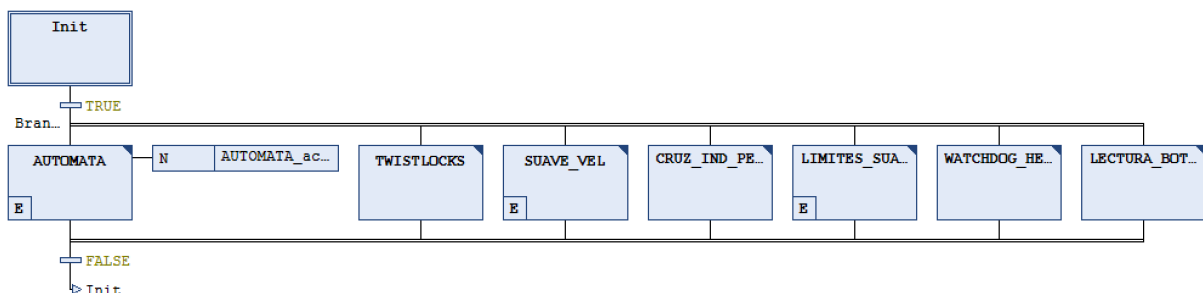


Figura 30: CODESYS - Diagrama general de control supervisor.

A diferencia de Stateflow, se debe agregar un estado paralelo adicional **LECTURA BOTONES**, el cuál se encarga exclusivamente de detectar flancos de subida de los botones presionados por el usuario.

El autómata principal se desarrolla de forma similar, y se encarga de alternar entre los modos de

Stand By, Manual, Automático, Errores y Recuperación.

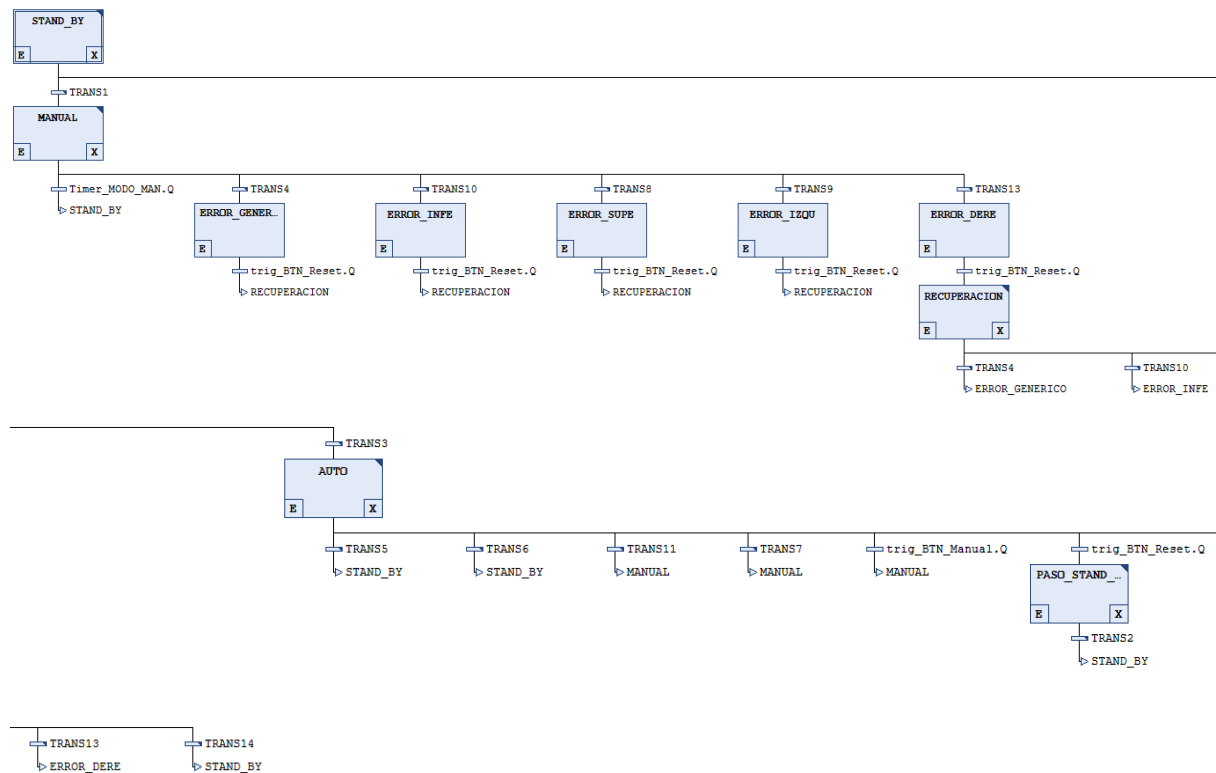
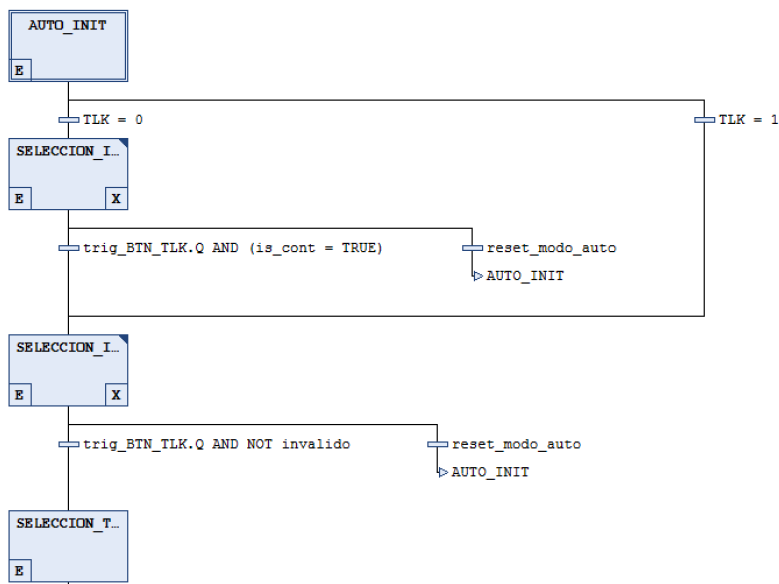


Figura 31: CODESYS - Supervisor - Autómata principal.

Para el modo automático se tuvo que modificar ligeramente las transiciones. En SFC, si se interrumpe un paso y salimos del modo automático, al volver al mismo no comienza por el paso inicial, sino que retoma en el paso interrumpido. Para evitar este comportamiento indeseado se agregó una variable adicional que permite transicionar al paso inicial. Esto no ocurre en Stateflow, ya que al entrar el modo automático éste siempre comienza a ejecutarse desde el estado inicial. En la siguiente imagen se puede ver la implementación utilizada:



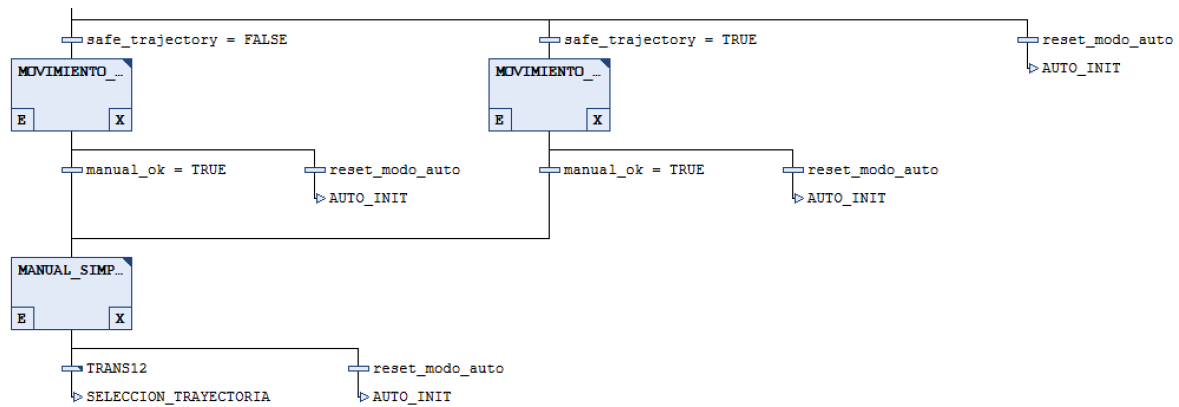


Figura 32: CODESYS - Supervisor - Autómata principal - Auto.

Sin modificaciones notorias, se replica el bloque encargado de la lógica de los Twistlocks. Como es un SFC cíclico, el último paso regresa al inicial al cumplir su transición:

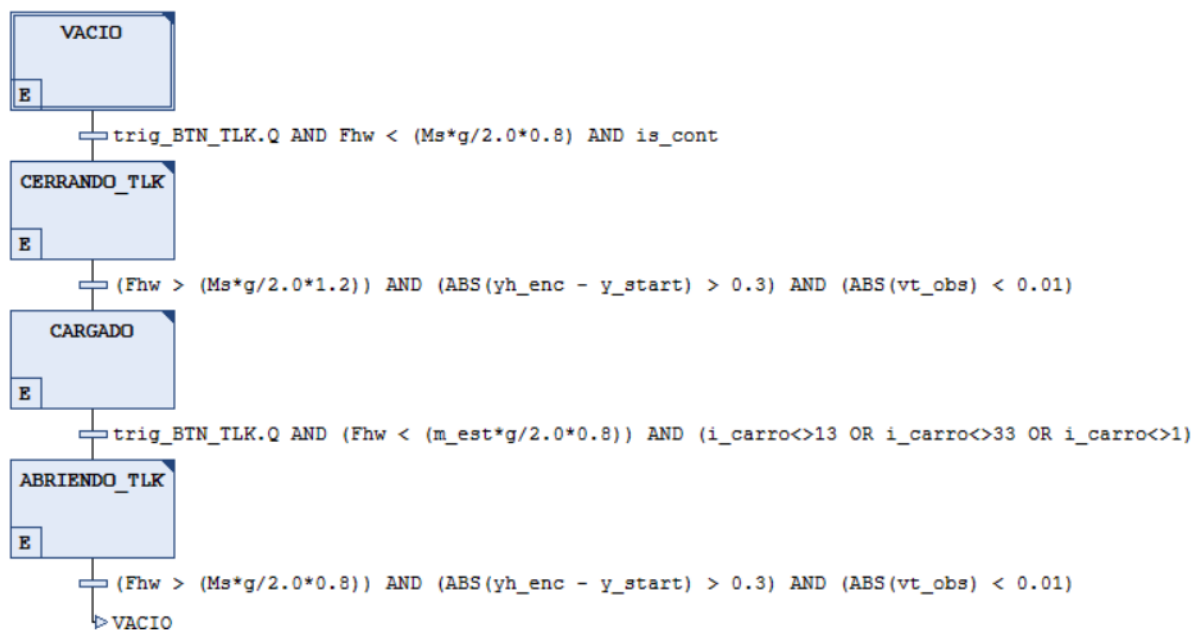


Figura 33: CODESYS - Supervisor - Lógica de Twistlocks.

De la misma forma, se utilizan las entradas de los sensores para calcular la posición discreta del carro y actualizar el perfil de obstáculos, información que es fundamental para la generación de trayectorias seguras en modo automático. También se actualiza la posición de la cruceta:

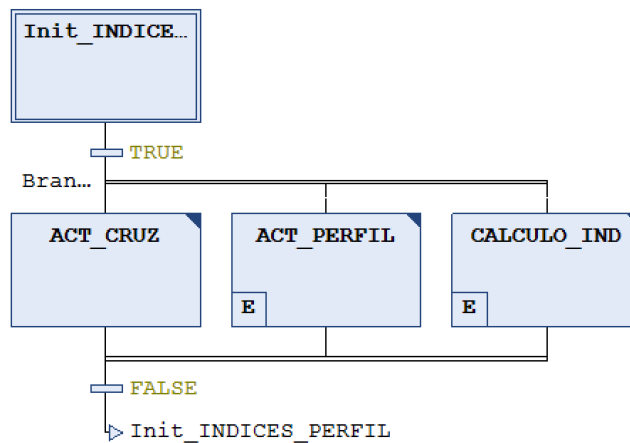


Figura 34: CODESYS - Supervisor - Cruceta Índices Perfil.

Los bloques de *Suavizador Velocidad*, *Límites Suave* y *Watchdog Heartbeat* se mantienen prácticamente iguales, desarrollados en ST directamente debido a su sencillez.

6.3. Control de Seguridad (Nivel 0) en CODESYS

Para el control de seguridad se mantuvo el mismo diseño, esta vez con dos procesos paralelos, uno para la máquina de estados de seguridad y otro para el watchdog. La lógica de ambos procesos se mantuvo prácticamente igual, con pequeñas modificaciones para adaptarse a la sintaxis y estructura de SFC.

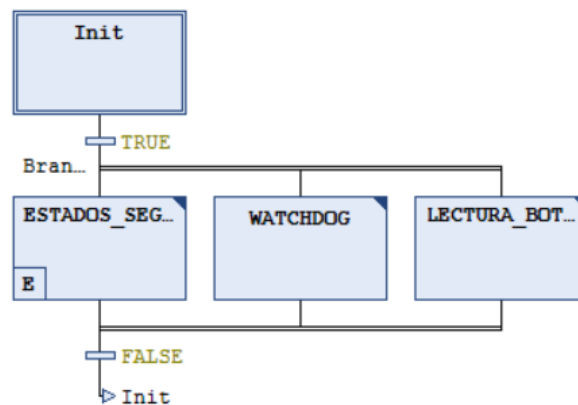


Figura 35: CODESYS - Seguridad - Diagrama general del control de seguridad.

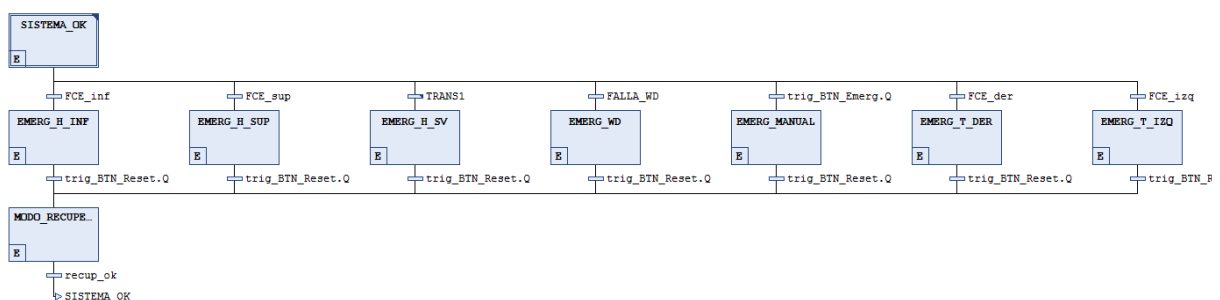


Figura 36: CODESYS - Seguridad - Estados de Seguridad.

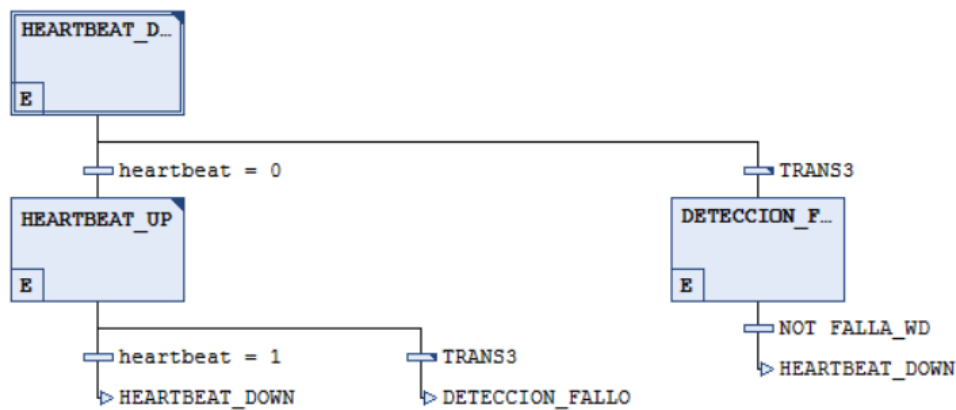


Figura 37: CODESYS - Seguridad - Watchdog.

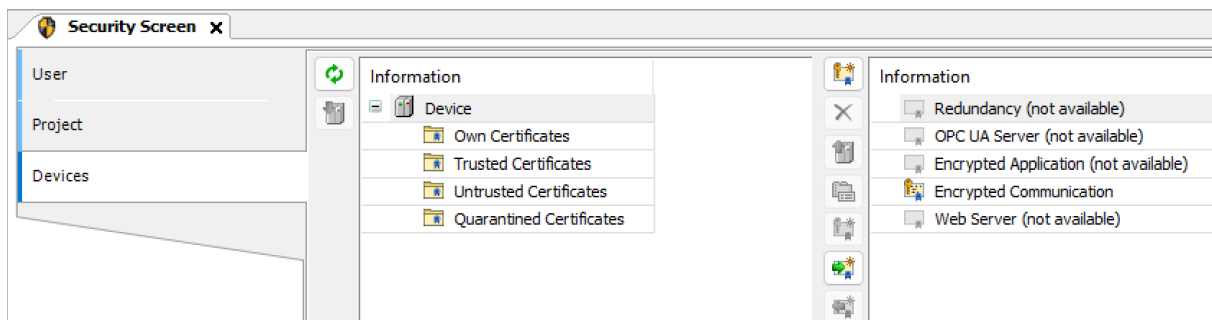
6.4. Comunicación OPC Simulink/CODESYS

Para poder comunicar el PLC virtual de CODESYS con Simulink, se utilizó el protocolo de comunicación industrial OPC-UA. La versión de CODESYS utilizada (CODESYS V3.5 SP21 Patch 2) cuenta con la posibilidad de montar un servidor OPC-UA y de seleccionar qué variables del programa se quieren exponer a través de este protocolo. Por otro lado, Matlab R2025b cuenta con un cliente OPC-UA que permite conectarse a este servidor y leer/escribir las variables expuestas en tiempo real.

Para permitir una conexión adecuada, se modificó algunos ajustes del dispositivo que hace de servidor. Se configuró sin encriptación, y la configuración de Device Security Settings se ve de la siguiente forma:

Device Security Settings	
Setting	Value
[-] CmpOPCUAServer	
CommunicationPolicy	POLICY_BASIC256SHA256
CommunicationMode	ALL
Activation	ACTIVATED
UserAuthentication	ENABLED
AllowUserPasswordOnPlaintext	NO
EnableCRLChecks	NO
EnableSelfSignedCertBackwardInteroperability	YES
CreateWithCAFlag	NO
DeactivateApplicationAuthentication	YES
DeactivateSecurityPolicy	
ApplicationName	OPCUAServer@Nahuel
CompanyOrOrganizationName	
City	
State	
Country	
CertificateIpAddresses	
[+] CmpOpenSSL	
[-] CmpUserMgr	
UserLogin_AuthenticationType	AUTO_NEGOTIATE
UserLogin_RSAKeyLen	2048
UserMgmtEnforce	NO
UserMgmtAllowAnonymous	YES
UserMgrEditTimeout	300

También se deshabilitó el certificado de OPC-UA en Security Screen, y se movió a Trusted el certificado que genera Matlab al intentar conectarse por primera vez:



Estos ajustes permiten a Simulink conectarse al servidor OPC-UA de CODESYS sin problemas, y leer/escribir las variables expuestas en tiempo real. Sin embargo, lo hace de forma anónima y sin seguridad alguna, lo cual no es recomendable para un entorno real, pero para fines de simulación y pruebas es suficiente.

Debido a la naturaleza de la comunicación OPC, cada lectura o escritura de una variable implica un proceso de comunicación cliente-servidor que introduce un retardo significativo. Este retardo se vuelve especialmente problemático cuando se intenta realizar lecturas/escrituras a alta frecuencia, lo cual es necesario para el control supervisor y más aún para el control regulatorio. OPC es un estandar de tipo TCP/IP, el cuál está diseñado para sistemas SCADA, con retardos de comunicación de 500 ms o más, y no para sistemas de control en tiempo real.

Para mitigar un poco este problema, se decidió implementar bloques de comunicación que se ejecuten a diferentes tiempos de muestreo. Esto enlentece la respuesta del modelo, causando delays peligrosos para la seguridad, pero permite que el sistema funcione de forma estable. El control regulatorio se ejecuta a 100 ms, el control supervisor y de seguridad a 500 ms. Para no tener errores con el heartbeat del watchdog, su variable corre al tiempo de muestro original de 20 ms. De esta forma, se reduce la cantidad de lecturas/escrituras por segundo, disminuyendo la carga en la red y permitiendo Simulink pueda correr a 1s por segundo de simulación.

Luego de algunas pruebas y reajustes, se mejoró significativamente la fluidez de la simulación, manteniendo una comunicación efectiva entre Simulink y CODESYS. Y se ha podido comprobar de forma satisfactoria que el controlador responde de la misma manera que el modelado en Simulink.

A continuación, los bloques de escritura/lectura de las variables de control regulatorio, supervisor y seguridad, respectivamente:

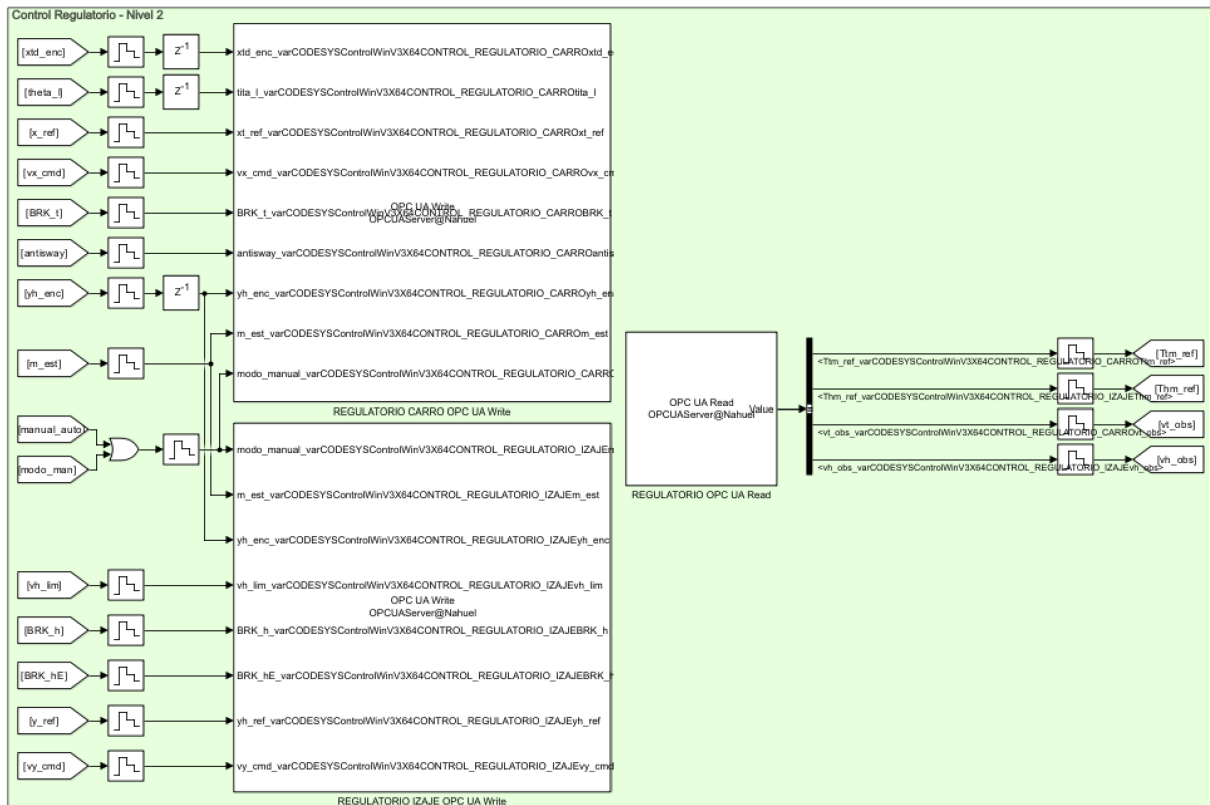


Figura 38: Simulink - Cliente OPC-UA - Control Regulatorio.

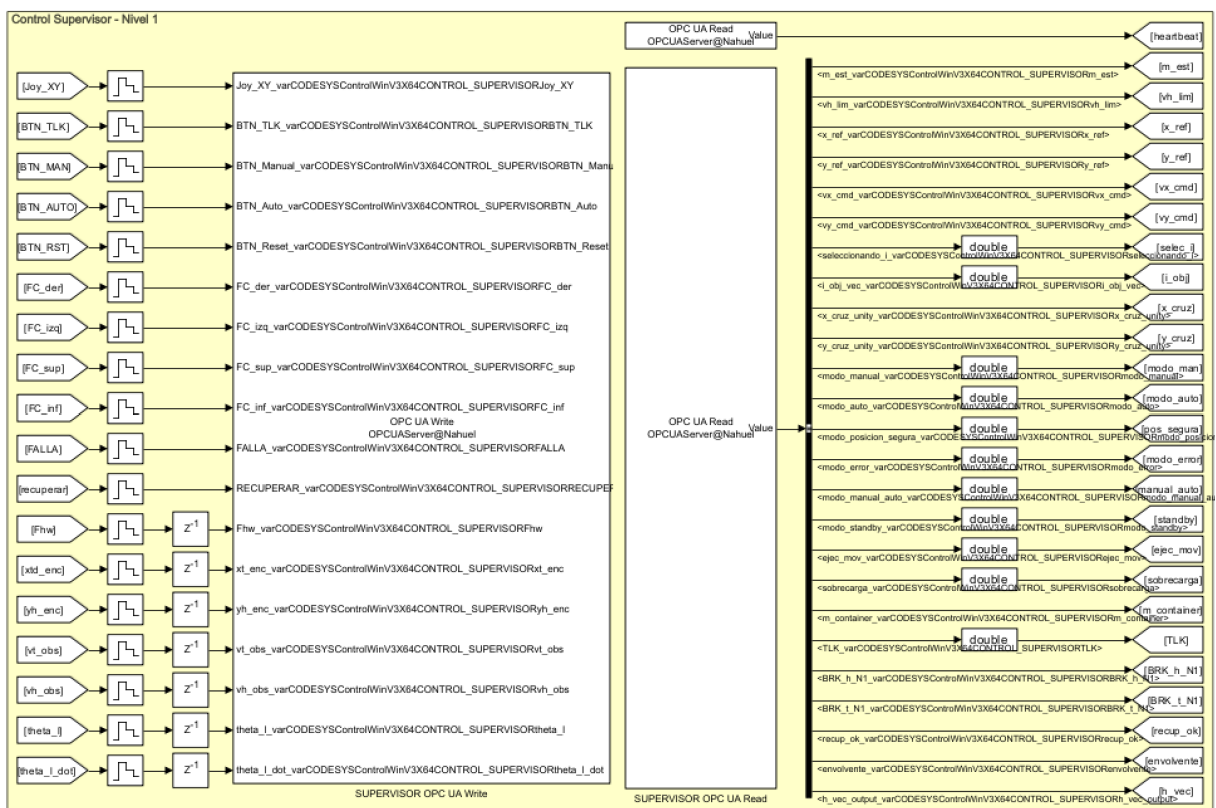


Figura 39: Simulink - Cliente OPC-UA - Control Supervisor.

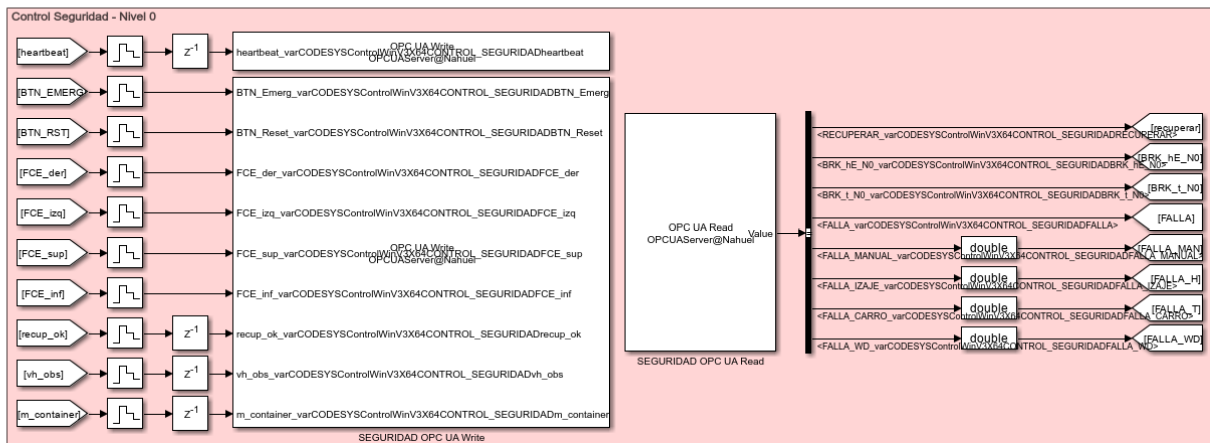


Figura 40: Simulink - Cliente OPC-UA - Control de Seguridad.

6.5. Comunicación UDP Simulink/CODESYS

Debido al retardo que se produce en la comunicación, consideramos que OPC no es un protocolo adecuado para la comunicación entre la simulación y el controlador. Para probar esta hipótesis, se implementó una comunicación alternativa utilizando el protocolo UDP, que es un protocolo de comunicación sin conexión y de baja latencia. Esto permite a la simulación correr en tiempo real, sin experimentar retrasos significativos en la comunicación. Se implementó para el control supervisor y de seguridad, dejando de lado el control regulatorio, ya que requiere una mayor velocidad de comunicación que incluso con UDP es difícil de mantener.

La implementación UDP es un poco más compleja, requiere el desarrollo de un protocolo de empaquetado y desempaquetado de datos, así como la sincronización entre los sistemas. Se creó por lo tanto dos programas adicionales en CODESYS, uno para enviar los datos a Simulink (COM_UDP_TX), y otro para recibir las variables desde Simulink (COM_UDP_RX).

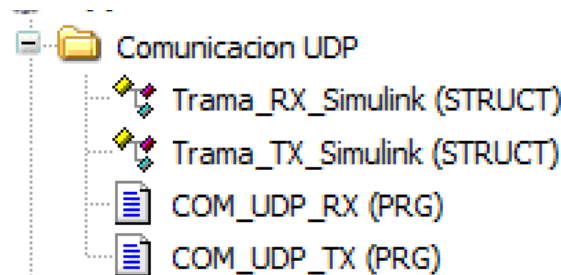


Figura 41: Nuevas funciones de comunicación UDP en CODESYS.

Los archivos de trama son de tipo DUT (Data Unit Type), y se definen como estructuras que organizan los datos a enviar. El orden es vital, ya que por UDP solo se envían bytes en un vector, y el receptor debe conocer el orden y tipo de cada dato para poder desempaquetarlo correctamente.

Los programas COM se encargan de repartir los valores de las variables a enviar o recibir a sus respectivas posiciones en la trama, y luego utilizan librerías con variables de envío o recepción.

```

1  PROGRAM COM_UDP_RX
2  VAR
3      Servidor_UDP : NBS.UDP_Peer;
4      Recibir_UDP  : NBS.UDP_Receive;
5      Buffer_RX     : ARRAY[0..263] OF BYTE;
6      Datos_In     : Trama_RX_Simulink;
7      Puerto       : UINT := 25000;
8      Ultimo_Contador : LREAL := -1.0;
9  END_VAR

10 // 1. Mantener el servidor escuchando siempre
11 Servidor_UDP(xEnable:= TRUE, uiPort:= Puerto);
12
13 // 2. Escuchar el puerto
14 Recibir_UDP(xEnable := Servidor_UDP.xActive, hPeer := Servidor_UDP.hPeer,
15   pData := ADR(Buffer_RX), szSize := SIZEOF(Buffer_RX)
16 );
17
18 // 3. Si llegaron datos, se procesan
19 IF (Datos_In.contador <> Ultimo_Contador) AND (Recibir_UDP.szCount >= SIZEOF(Datos_In))
20 THEN Ultimo_Contador := Datos_In.contador;
21
22   SysMem.SysMemCpy(pDest := ADR(Datos_In), pSrc := ADR(Buffer_RX), udiCount := SIZEOF(Datos_In));
23
24   // --- Seguridad ---
25   CONTROL_SEGURIDAD.BTN_Reset := TO_BOOL(Datos_In.BTN_Reset);
26   CONTROL_SEGURIDAD.BTN_Emerg := TO_BOOL(Datos_In.BTN_Emerg);
27   CONTROL_SEGURIDAD.FCE_der   := TO_BOOL(Datos_In.FCE_der);
28   CONTROL_SEGURIDAD.FCE_izq   := TO_BOOL(Datos_In.FCE_izq);
29   CONTROL_SEGURIDAD.FCE_sup   := TO_BOOL(Datos_In.FCE_sup);
30   CONTROL_SEGURIDAD.FCE_inf   := TO_BOOL(Datos_In.FCE_inf);
31   CONTROL_SEGURIDAD.recup_ok  := TO_BOOL(Datos_In.recup_ok);
32   CONTROL_SEGURIDAD.vh_obs    := Datos_In.vh_obs;
33   CONTROL_SEGURIDAD.m_container := Datos_In.m_container;
34   CONTROL_SEGURIDAD.heartbeat := TO_INT(Datos_In.heartbeat);
35
36   // --- Supervisor ---

```

Figura 42: Función de Recepción UDP en CODESYS.

```

1  PROGRAM COM_UDP_TX
2  VAR
3      Cliente_UDP : NBS.UDP_Peer;
4      Enviar_UDP  : NBS.UDP_Send;
5      Buffer_TX     : ARRAY[0..1023] OF BYTE;
6      Datos_Out    : Trama_TX_Simulink;
7      IP_Destino   : NBS.IP_ADDR := (sAddr := '127.0.0.1');
8      Puerto       : UINT := 25001;
9      k            : INT;
10 END_VAR

11 // 1. Empaquetado de resultados actuales de los controladores
12 // --- Seguridad ---
13 Datos_Out.BRK_hE_NO := TO_LREAL(CONTROL_SEGURIDAD.BRK_hE_NO);
14 Datos_Out.BRK_t_NO  := TO_LREAL(CONTROL_SEGURIDAD.BRK_t_NO);
15 Datos_Out.FALLA_CARRO_DER := SEL(CONTROL_SEGURIDAD.FALLA_CARRO_DER, 0.0, 1.0);
16 Datos_Out.FALLA_CARRO_IZQ := SEL(CONTROL_SEGURIDAD.FALLA_CARRO_IZQ, 0.0, 1.0);
17 Datos_Out.FALLA_IZAJE_INF := SEL(CONTROL_SEGURIDAD.FALLA_IZAJE_INF, 0.0, 1.0);
18 Datos_Out.FALLA_IZAJE_SUP := SEL(CONTROL_SEGURIDAD.FALLA_IZAJE_SUP, 0.0, 1.0);
19 Datos_Out.FALLA_IZAJE_SV  := SEL(CONTROL_SEGURIDAD.FALLA_IZAJE_SV, 0.0, 1.0);
20 Datos_Out.FALLA_MANUAL    := SEL(CONTROL_SEGURIDAD.FALLA_MANUAL, 0.0, 1.0);
21 Datos_Out.FALLA_WD        := SEL(CONTROL_SEGURIDAD.FALLA_WD, 0.0, 1.0);
22
23 // --- Supervisor ---
24 Datos_Out.m_est := CONTROL_SUPERVISOR.m_est;

```

Figura 43: Función de Transmisión UDP en CODESYS.

En Simulink, al igual que con la comunicación con la animación, se implementaron bloques de lectura/escritura de UDP, permitiendo enviar y recibir datos en tiempo real sin los retrasos de OPC.

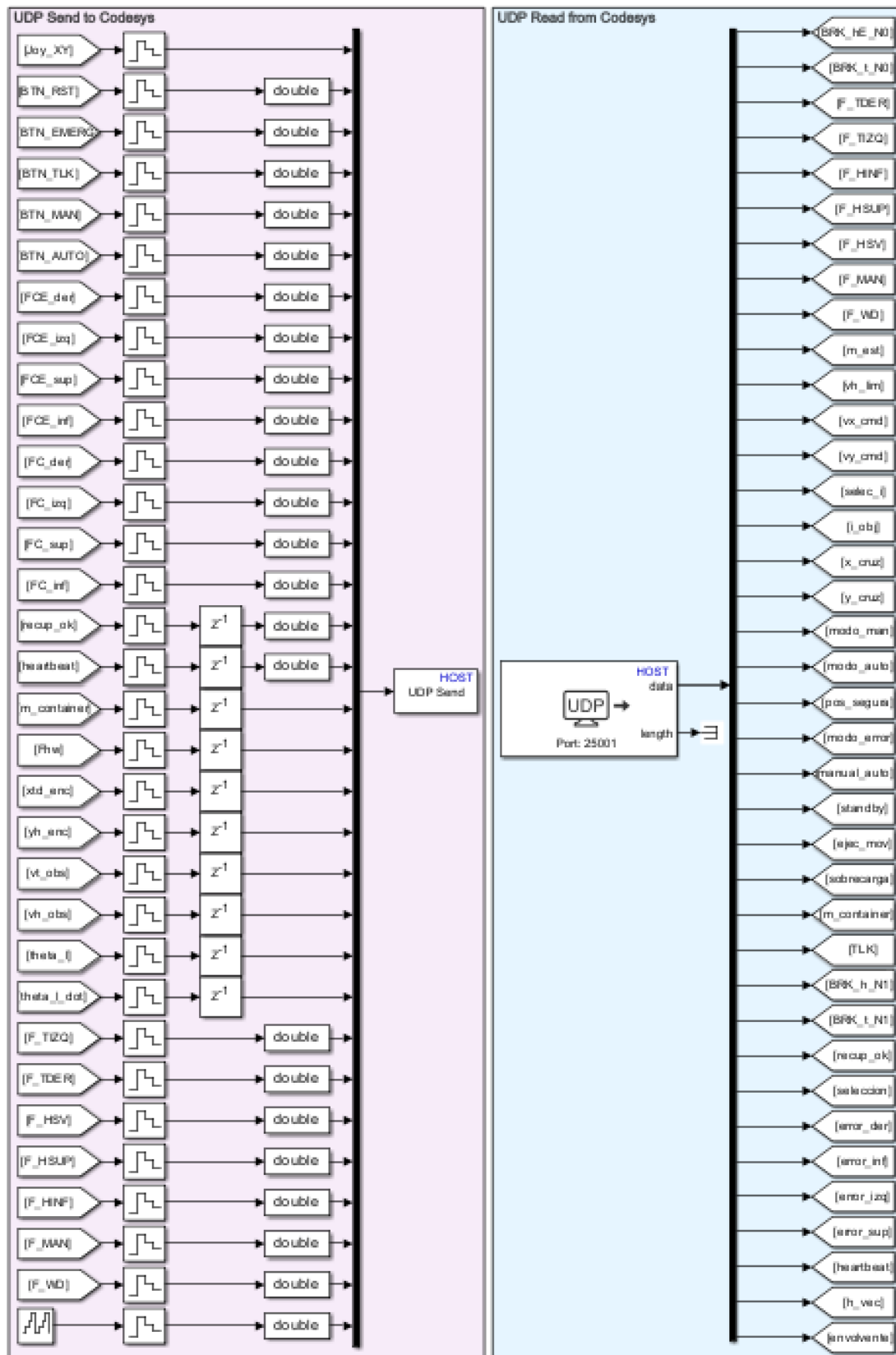


Figura 44: Diseño UDP en Simulink.

Para lograr una sincronización adecuada, se debe programar una tarea en CODESYS que ejecute la función de lectura de datos más rápido que la de envío, para evitar pérdida de información. En este caso, se programó a 10 ms, mientras que el bloque de escritura se ejecuta a 20 ms. Además, el orden de ejecución también es vital, por lo que la prioridad más elevada es para la lectura (luego el control de seguridad, el control supervisor, y por último el envío de datos). En Simulink es necesario activar *pacing* a tiempo real, es decir, 1 segundo.

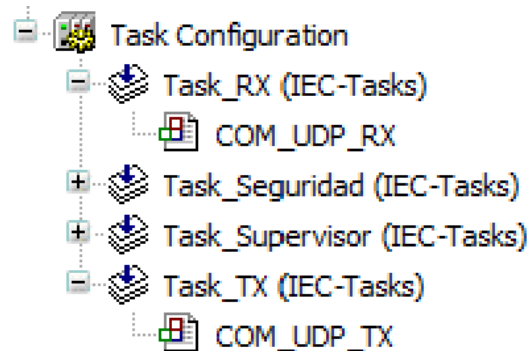


Figura 45: Task Configuration en CODESYS.

A continuación se muestra una captura de la simulación funcionando a través de UDP:

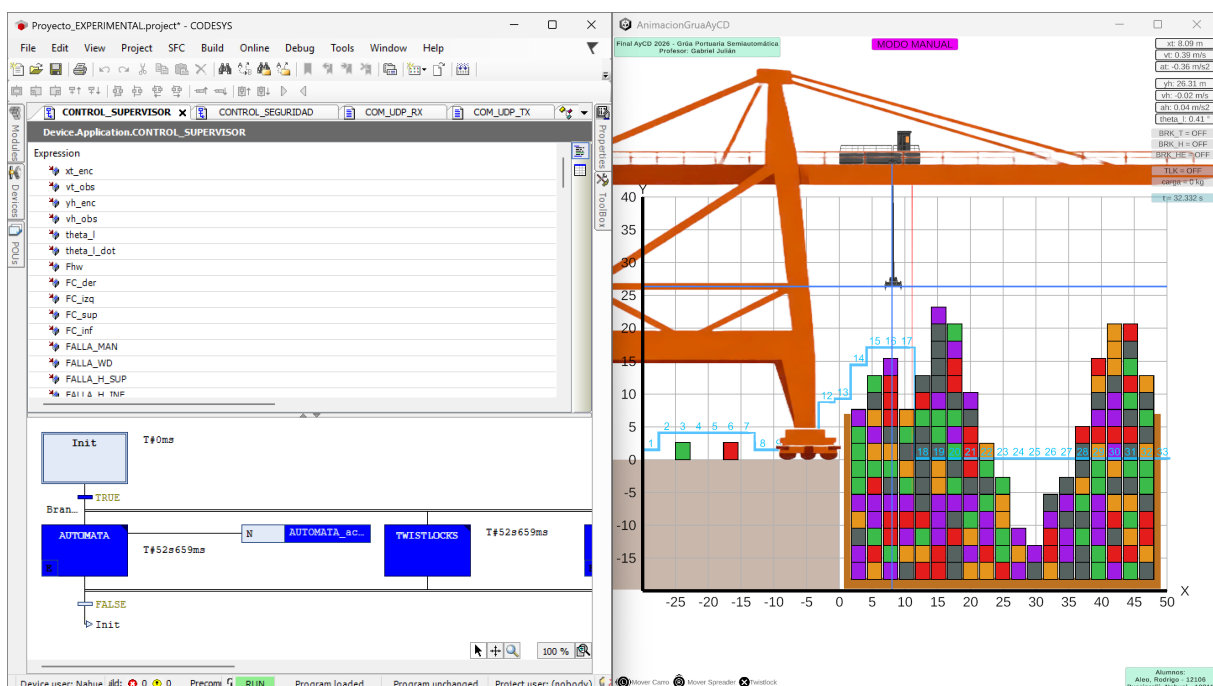


Figura 46: Simulación funcionando a través de UDP.

7. Generación Automática de Código ST

Simulink ofrece una herramienta llamada PLC Coder, que ofrece la capacidad de traducir automáticamente diagramas Stateflow a Texto Estructurado (ST) bajo la norma IEC 61131-3. Se procedió a analizar el código ST resultante de esta autogeneración. Se puede observar que el traductor implementa el autómata de forma “plana” y monolítica. Para lograrlo, recurre a una estructura de control **CASE** global que evalúa una variable entera representativa del estado actual,

combinada con una densa red de sentencias IF/ELSIF anidadas para resolver las transiciones y acciones correspondientes.

A nivel de eficiencia computacional y carga de CPU, el rendimiento del código autogenerated es comparable al de una implementación manual. Los compiladores modernos de los PLC optimizan ambas estructuras a un código máquina de ejecución sumamente rápida, por lo que el tiempo de ciclo del autómata no se ve significativamente alterado por la elección de uno u otro método.

Sin embargo, la divergencia crítica entre ambos enfoques radica en la legibilidad, la mantenibilidad y el diagnóstico de fallas en el entorno industrial. El código ST autogenerated carece de trazabilidad visual; resulta ser una estructura de difícil interpretación para los operarios de planta. Por este motivo, en el proyecto se optó por utilizars solamente la traducción manual, utilizando SFC para la arquitectura general de los autómatas y reservando el Texto Estructurado exclusivamente para las acciones matemáticas subyacentes.

Esta estrategia mantiene la potencia de cálculo del ST para las leyes de control cinemático, mientras que el entorno SFC proporciona un diagnóstico más visual. A continuación se muestran fragmentos del código autogenerated para el control de seguridad, evidenciando la complejidad y falta de claridad que presenta:

```
1  FUNCTION_BLOCK Control
2  VAR_INPUT
3      ssMethodType: SINT;
4      BTN_Emerg: BOOL;
5      FCE_der: BOOL;
6      FCE_sup: BOOL;
7      vh_obs: LREAL;
8      FCE_izq: BOOL;
9      FCE_inf: BOOL;
10     BTN_Reset: BOOL;
11     heartbeat: LREAL;
12     m_container: LREAL;
13     recup_ok: LREAL;
14 END_VAR
15 VAR_OUTPUT
16
17 CASE ssMethodType OF
18     SS_INITIALIZE:
19         (* SystemInitialize for Chart: '<Root>/Control Nivel 0' incorporates:
20          *   Outport: '<Root>/FALLA'
21          *   Outport: '<Root>/FALLA_CARRO'
22          *   Outport: '<Root>/FALLA_IZAJE'
23          *   Outport: '<Root>/FALLA_MANUAL'
24          *   Outport: '<Root>/FALLA_WD'
25          *   Outport: '<Root>/RECUPERAR' *)
26         FALLA := 0.0;
27         RECUPERAR := 0.0;
28         FALLA_MANUAL := 0.0;
29         FALLA_IZAJE := 0.0;
30         FALLA_CARRO := 0.0;
31         FALLA_WD := 0.0;
32         WD_counter := 0.0;
33         vh_lim := 0.0;
34         is_active_c7_Control := 0;
```

```

72      CASE is_ESTADOS_SEGURIDAD OF
73      Control_IN_EMERGENCIA_CARRO:
74          (* Outport: '<Root>/BRK_hE_NO' *)
75          BRK_hE_NO := 1.0;
76          (* Outport: '<Root>/BRK_t_NO' *)
77          BRK_t_NO := 0.0;
78          FALLA_CARRO := 1.0;
79          FALLA := 1.0;
80          (* During 'EMERGENCIA_CARRO': '<S1>:26' *)
81          (* '<S1>:46:1' sf_internal_predicateOutput = 0 | (BTN_Reset); *)
82
83      IF BTN_Reset THEN
84          (* Transition: '<S1>:46' *)
85          is_ESTADOS_SEGURIDAD := c_Control_IN_MODO_RECUPERAC;
86          (* Outport: '<Root>/BRK_t_NO' *)
87          (* Entry 'MODO_RECUPERACION': '<S1>:41' *)
88          (* '<S1>:41:3' BRK_hE_NO = 1; *)
89          (* '<S1>:41:3' BRK_t_NO = 1; *)
90          BRK_t_NO := 1.0;
91          (* '<S1>:41:4' RECUPERAR = 1; *)
92          RECUPERAR := 1.0;
93      END_IF;
94
95      Control_IN_EMERGENCIA_IZAJE:
96          (* Outport: '<Root>/BRK_hE_NO' *)
97          BRK_hE_NO := 0.0;
98          (* Outport: '<Root>/BRK_t_NO' *)
99          BRK_t_NO := 1.0;
100          FALLA_IZAJE := 1.0;
101          FALLA := 1.0;
102          (* During 'EMERGENCIA_IZAJE': '<S1>:30' *)
103          (* '<S1>:45:1' sf_internal_predicateOutput = 0 | (BTN_Reset); *)
104
105      IF BTN_Reset THEN
106          (* Transition: '<S1>:45' *)
107          is_ESTADOS_SEGURIDAD := c_Control_IN_MODO_RECUPERAC;
108          (* Outport: '<Root>/BRK_hE_NO' *)
109          (* Entry 'MODO_RECUPERACION': '<S1>:41' *)
110          (* '<S1>:41:3' BRK_hE_NO = 1; *)

```

Figura 47: Código generado por PLC Coder - Fragmentos del control de seguridad.

8. Conclusiones

En este proyecto se logró atravesar las diferentes etapas del diseño de control, desde la modelación física y el desarrollo de controladores en Simulink, hasta la implementación en CODESYS y la integración con una aplicación de visualización en Unity. En ese sentido, fue un trabajo integral de control mecatrónico, porque no se quedó solo en la simulación ni solo en la programación del PLC, sino que conectó modelado, control, supervisión y validación visual dentro de un mismo flujo.

Uno de los aprendizajes más importantes fue el valor del modelo físico. Tener una base dinámica razonable permitió diseñar y ajustar mejor los controladores, pero también dejó claro que todo modelo tiene límites y simplificaciones. Considerar esas limitaciones desde el principio ayudó a interpretar mejor los resultados y a no asumir comportamientos ideales que después no se sostienen al pasar a una arquitectura industrial.

También fue clave el modelo de los controles, sobre todo por cómo se organizaron los distintos niveles. El uso de herramientas como las máquinas de estado permitió estructurar la lógica de forma más clara y robusta, facilitando la detección de condiciones de falla y la definición de respuestas seguras ante eventos críticos.

Un desafío concreto fue adaptar la lógica desarrollada en Stateflow al entorno de CODESYS. Pasar de un enfoque gráfico basado en estados y transiciones a una implementación en SFC y ST exigió reordenar la lógica, cuidar el manejo de eventos y temporizaciones, y validar que el comportamiento final se mantuviera equivalente al del modelo original.

El desarrollo de la interfaz gráfica en Unity aportó una representación visual en tiempo real muy útil para validar el comportamiento global de la grúa. Esta etapa permitió además incorporar y comprender la comunicación UDP entre Simulink y Unity, incluyendo el empaquetado y tratamiento de datos.

Como último comentario, esta misma comunicación UDP se pudo implementar con éxito entre Simulink y CODESYS, permitiendo un rendimiento superior de la simulación. Consideramos que podría ser una alternativa viable a OPC para este proyecto, aunque con sus propias complejidades.

9. Repositorio y Video Demostrativo

El proyecto completo, incluyendo los modelos de Simulink, el código en ST para CODESYS y la aplicación de Unity, se encuentra disponible en el siguiente [repositorio de GitHub](#).

Además, se ha preparado un video demostrativo que muestra el funcionamiento de la grúa en diferentes modos de operación, destacando la interacción entre los niveles de control y la visualización en Unity. El video se puede acceder a través del siguiente [enlace a Youtube](#).

Referencias

- [1] [Simocrane Sway Control System for STS](#). Siemens.
- [2] [Sway Control Technology - White Paper](#). CMCO.
- [3] [Anti-sway System for Ship-to-Shore Cranes](#).
- [4] [Crane Anti-Sway Control System with Sway Angle Feedback](#)
- [5] [CODESYS Help - Documentación oficial de CODESYS](#).